1. Background

1.1. Mathematical Notation

When discussing probabilities, random variables (RVs) are represented by capital letters X. Assignments to this RV are denotes as lowercase x.
Scalar quantities will be written as $x \in \mathbb{R}$, while vectors will be denoted as bold face $\mathbf{x} \in \mathbb{R}^D$. A density function of variable x will be denoted as $p(x)$. If the density is parameterized by a parameter θ we will write $p_\theta(x)$. If θ is a vector it will be written bold $\boldsymbol{\theta}$. If we are talking about the data-generating distribution, we will write $p_{\text{data}}(x)$ or simply $p_d(x)$.

When discussing data, a whole dataset is denoted as $\mathcal{X} = \{x_1, ..., x_N\}$ where N is the number of samples. Again scalar quantities will be written as $x \in \mathbb{R}$, and vectors as bold face $\mathbf{x} \in \mathbb{R}^D$.

The notation for partially observed points, $x = (x^o, x^m)$ with the observed features x^o, the missing features x^m and the matching split $w = (w_o, w_m)$ of the classifier, will be introduced here.

1.2. Probabilistic Modelling

At its core, **Machine Learning (ML)** aims to develop algorithms or programs that can learn from data to make informed decisions, predict future outcomes, or perform various inference tasks. One powerful approach to this, especially when we are uncertain about particular quantities –which is the case for most real-world scenarios– is **Probabilistic Modelling**. In Probabilistic Modelling, we use probabilities and probability densities to represent this uncertainty and employ the rules of probability theory to carry out inference tasks [1].

1.2.1. Probability Basics

To clarify these ideas, consider two random variables X and Y, that can assume values x and y. If we know how the probabilities are distributed over all possible pairs of

outcomes (x, y), we know the *joint probability distribution* $p(x, y)$. For *discrete* random variables, this joint distribution is described by a *probability mass function* (PMF). For *continuous* random variables, it can often be described by a *probability density function* (PDF). Since we will mostly deal with continuous random variables in this thesis, we will focus on densities going forward.

For any function to be a valid probability density it has to fulfill two properties. It has to be non-negative $p(x) \geq 0$ for all x and it has to be normalized so that the total area under the function equals 1, i.e. it has to integrate to 1 over the entire space: $\int_{-\infty}^{\infty} p(x)\, dx = 1$.

Having access to a valid joint density function $p(x, y)$, allows us –in principle– to perform basically all key tasks that we want to accomplish in machine learning. [1] We can sample from the distribution to generate new data points, for example to generate images. Moreover, we can predict by evaluating the density at a specific point and we can compute more complex inference tasks by using rules from probability theory, like the marginal probability (MAR) or the conditional probability (CON) [1].

The marginal probability function $p(x)$ is the probability of observing x regardless of the value of y. It is computed by integrating over all possible values of y:

$$p(x) = \int p(x, y)\, dy$$

The conditional probability function $p(x|y)$ is the probability of observing x given y. It is computed by dividing the joint probability by the marginal probability of y:

$$p(x|y) = \frac{p(x, y)}{p(y)}$$

1.2.2. Modelling a Density

Assume that we are given data $\mathcal{X} = \{x_1, ..., x_N\}$ that was generated by an unknown density function $p_{\text{data}}(x)$.
In this work, we always assume $x_i \overset{\text{iid}}{\sim} p_{\text{data}}$, which means a sample x_i is *independently* and *identically* distributed (iid) according to p_{data}.

In a real-world scenario, we typically don't have access to $p_{\text{data}}(x)$ and finding it can be seen as the "holy grail" of machine learning, because if it was known to us all inference tasks would reduce to simply applying the rules of probability theory [1]. However since we don't know $p_{\text{data}}(x)$, we create a parameterized model $p_\theta(x)$ and, using the

[1] Note that wile probabilistic inference is conceptually simple, it is often computationally difficult in practice.

data, we tune the parameter θ so that $p_\theta(x)$ approximates $p_{\text{data}}(x)$ as closely as possible.

One of the most basic yet widely used methods for learning such a model is Maximum Likelihood Estimation (MLE) [2]. In principle, MLE is straightforward: we tune θ so that the likelihood of the observed data is maximized under the model. Specifically we *maximize the likelihood* (the output of the model) of each data point in $\mathcal{X}$ with respect to θ, as expressed in Equation 1.1:

$$\max_\theta \{L(\theta) = \prod_{i=1}^{N} p_\theta(x_i)\} \tag{1.1}$$

Where $L(\theta)$ is usually referred to as the *Likelihood Function*.

It is noteworthy that instead of modelling $p_\theta(x)$, most of the time we want to model $\log p_\theta(x)$ because it typically is easier to work with. So instead of maximizing the Likelihood we maximize the Log-Likelihood, turning Equation 1.1 into the following:

$$\max_\theta \{LL(\theta) = \log \prod_{i=1}^{N} p_\theta(x_i) = \sum_{i=1}^{N} \log p_\theta(x_i)\} \tag{1.2}$$

Here $LL(\theta)$ is now referred to as the *Log-Likelihood Function*.

After learning a model using MLE we can again perform inference using the rules of probability.

1.2.3. Tractability vs. Expressiveness

Before continuing with different ways of constructing such a model, we want to introduce two key concepts that are often used to discuss these models.

1. **Expressiveness**
   Expressiveness refers to the ability of a model to approximate different sets of distributions $p_{\text{data}}(x)$ to a high degree. If model A can approximate a set of distributions well, but model B can also model this set to the same degree plus additional distributions, then B would be more expressive than A. However normally this term is used as in expressive efficiency, so how complex a model must be to be expressive. For instance, if both models A and B can approximate the same set of distributions with equal effectiveness, but model B does so with significantly less complexity, then model B is regarded as more expressive (efficient) [1].

2. **Tractability**
   A probabilistic model is called tractable with respect to an inference task, if the model can complete this task exactly and efficiently. *Exactly* in this case means without relying on approximation and *efficiently* means in polynomial time with respect to the model size [1].

In recent years expressive capability has increased considerably through models based on neural networks like Generative-Adversarial Networks (GANs) [3], Variational Autoencoder (VAE) [4] and many others. However while these models excel in generating very realistic images or compelling text, they lack in tractability for all except the simplest inference tasks [1].

On the other hand there are many, mostly older, less complex models like Gaussian Mixture Models (GMMs), Hidden Markov Models (HMMs) and so on, that lack in expressiveness and only work well enough for simple data but can compute many inference tasks tractably [2].

1.3. Missing Data

This section will introduce the missingness mechanisms (MCAR, MAR, MNAR) and the usual ways of handling missing features (dropping points, mean imputation, multiple imputation, marginalisation), which motivate the expected prediction as the goal.

1.4. Logistic Regression

This section will introduce logistic regression $p(y \mid x) = \sigma(w^T x)$, its log loss with L_2 penalty and how it is optimised.

1.5. Gaussian Mixture Model

A Gaussian Mixture Model (GMM) [2] is a well-known probabilistic model that will serve as good groundwork going forward.

The basic idea of a GMM is that a single Gaussian density cannot model complex distributions very well. However, a weighted sum of many different Gaussians, a so-called ”mixture” can be an expressive density estimator.

The probability density function of a multivariate GMM is given by Equation 1.3.

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{1.3}$$

In this equation, K is the number of Gaussian components in the mixture, and π_k is the mixture weight associated with the k-th component, where $\sum_{k=1}^{K} \pi_k = 1$ and $\pi_k \geq 0$, ensuring that the model outputs a normalized density.

$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ represents one single Gaussian component with mean $\boldsymbol{\mu}_k$ and covariance matrix $\boldsymbol{\Sigma}_k$. This component is calculated using the well known multivariate gaussian density function:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \tag{1.4}$$

.

Furthermore $\boldsymbol{\theta}$ refers to the parameter vector $\boldsymbol{\theta} = \{\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}\}$, where $\boldsymbol{\pi} = \{\pi_1, ..., \pi_K\},\ \boldsymbol{\mu} = \{\boldsymbol{\mu}_1, ..., \boldsymbol{\mu}_K\},\ \boldsymbol{\Sigma} = \{\boldsymbol{\Sigma}_1, ..., \boldsymbol{\Sigma}_K\}$.

We call $\boldsymbol{\theta}$ the learnable parameters and K the hyperparameter of the model.

How a GMM is conditioned on the observed features is shown in Section 2.3.1, where it is needed for the characteristic function.

1.6. Probabilistic Circuits

Probabilistic Circuit (PC) [1] is an umbrella term for probabilistic models that can perform many inference tasks tractably but can be highly expressive at the same time. In general, this is achieved by only introducing complexity in a structured manner. More specifically, for a PC to be valid it has to adhere to certain structural properties, but more on that later.

Prominent members of the PC class include Cutset Networks [5], Probabilistic Sentential Decision Diagrams (PSDDs) [6] and others. In this work, we focus on a specific type of PC, which is often also called a Sum-Product Network (SPN) [7].

In principle, a PC is a structured neural network that consists of leaf nodes, sum nodes and product nodes. A PC then recursively computes weighted mixtures (sum nodes) and factorizations (product nodes) of simple input distributions (leaf nodes). These inputs can be any probability distribution like Gaussians, Binomials, Categoricals etc., however most of the time we will talk about Gaussians leafs [1]. Note that if the leaf

distributions are normalized density functions and the weights of the sum nodes of the PC are normalized correctly, then the whole PC also models a normalized density [1].

Considering this, one could notice that a simple instantiation of a PC is the aforementioned Gaussian Mixture Model, where one sum node mixes two leaf nodes. Intuitively, PCs overall can fundamentally be seen as *deep* (Gaussian) Mixture Models. This simple PC can be graphically depicted as in Figure 1.1, where the two nodes at the bottom represent input distributions over the RV X_1 and the top node represents the mixture.

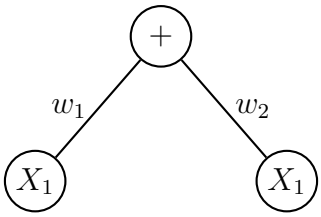


Figure 1.1.: Simplest PC - GMM

Furthermore more complex PCs can be created by hierarchically stacking sum nodes and product nodes. An only slightly more complex PC with a product node and two sum nodes can be seen in Figure 1.2.

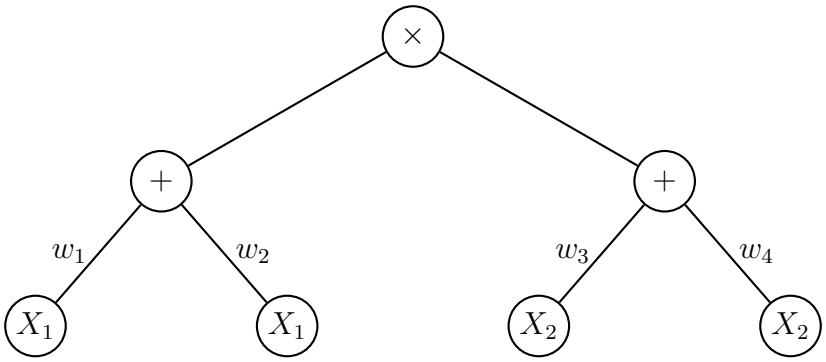


Figure 1.2.: PC: A product of two one-dimensional GMMs

Although this is still very very basic, it is a great starting point to introduce and make sense of two central structural properties that allow PCs to be expressive and tractable.

Definition 1 (Scope)**.** If a PC P models the joint distribution of a set of variables $\mathbf{X}$, each node of P models a distribution over a subset of $\mathbf{X}$. This subset is called the scope of the node. For a leaf node the scope is the input variable to that leaf, for all other nodes the scope is the union of its children's scopes. So the root node always has scope $\mathbf{X}$ [1].

Definition 2 (Smoothness)**.** A sum node is *smooth* if all its inputs have identical scopes. A PC is smooth if all its sum nodes are smooth [1].

Definition 3 (Decomposability)**.** A product node is *decomposable* if all it's input scopes do not share variables. A PC is decomposable if all of its product nodes are decomposable [1].

In simple terms this basically means that sum nodes are only allowed to have inputs over the same variables and product nodes are only allowed to have inputs over different variables. The two depicted PCs are smooth and decomposable.

There are more structural properties that a PC can fulfill, however these two already guarantee tractable computation of the important marginal (MAR) and conditional (CON) queries [1], that we discussed in Section 1.2.1. This is already sufficient for many scenarios. Generally the more structure a PC has, which means the more structural properties it must fulfill, the more inference tasks it can compute tractably.

We refer the reader to [1] for why these properties guarantee tractable inference and further discussion on other properties.

Using a PC in a real-world scenario then would entail first deciding on a structure (i.e., which properties and leaf distributions to use and how the nodes should be arranged in the graph) and then doing a Maximum Likelihood Estimation, where θ is composed of the weights of the sum nodes and the parameters of the leaf distributions (e.g., means and variances for a Gaussian). This optimization is usually done via Gradient Descent or Expectation-Maximization (EM).

This section will also explain why the expectation of a function that factorises over the features can be pushed down to the leaves, and the RAT-SPN structure used in the experiments. How a PC marginalises missing features is shown in Section 2.5.

1.7. Numerical Integration

1.7.1. Monte Carlo

1.7.2. Gaussian Quadrature

Gauss–Hermite Quadrature

Golub–Welsch Algorithm

For a distribution without a standard rule, the *Golub–Welsch algorithm* [8] produces the n-point Gaussian quadrature nodes and weights from its first $2n$ moments $\mu_k := \mathbb{E}[x^k]$, $k = 0, \dots, 2n-1$. Form the $n \times n$ Hankel moment matrices

$$H_{ij} = \mu_{i+j}, \qquad H'_{ij} = \mu_{i+j+1}, \quad i, j = 0, \dots, n-1.$$

Compute the Cholesky factor $H = R^\top R$ and the Jacobi (tridiagonal) matrix

$$J = R^{-\top} H' R^{-1}, \quad \text{symmetrised as } J \leftarrow \tfrac{1}{2}(J + J^\top).$$

Eigendecompose $J = V \Lambda V^\top$. Then the quadrature nodes and weights are

$$x_j = \lambda_j, \qquad w_j = \mu_0 \, v_{0j}^2, \quad j = 1, \dots, n,$$

where λ_j are the eigenvalues and v_{0j} is the first component of the j-th eigenvector. The resulting rule integrates polynomials exactly up to degree $2n-1$.

1.7.3. Effect of Dimensionality

Why quadrature is only good in low dimensions.

1.8. Characteristic Function

1.9. Pólya–Gamma Distribution

2. Methods

2.1. Core Idea

2.1.1. Setup

Consider a logistic regression, with missing features in the observation.
Let $x := (x^o, x^m) \in \mathbb{R}^d$. Then the logistic regression is defined as

$$p(y \mid x) := \sigma(w^T x), \qquad w \in \mathbb{R}^d.$$

Additionally we have a PC that models the density $p(x)$.

2.1.2. Goal

Compute the prediction in a probabilistically principled manner using

$$p(y \mid x^o) := \mathbb{E}_{p(x^m \mid x^o)} \left[\sigma \left(w^T x \right) \right].$$

If it were not for the non-linear sigmoid function σ, we would be able to calculate this in a smooth and decomposable PC, by pulling the Expectation -which is linear- into the leaf nodes. To tackle this we propose to use two tricks.

2.1.3. Trick 1: Pólya-Gamma

To replace the sigmoid function we use the PG-Sigmoid identity, shown in [9], that states:

$$\sigma(z) = \frac{1}{2} \exp\left(\frac{1}{2} z\right) \; \mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)} \left[\exp\left(-\frac{1}{2} \gamma z^2 \right) \right]$$

for $z \in \mathbb{R}$.

If we set $z := w^T x$, and insert into our expectation, we get:

$$\begin{aligned} p(y \mid x^o) &= \mathbb{E}_{p(x^m|x^o)}\left[\sigma(z)\right] \\ &= \mathbb{E}_{p(x^m|x^o)}\left[\frac{1}{2}\exp\left(\frac{1}{2}z\right)\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(-\frac{1}{2}\gamma z^2\right)\right]\right] \\ &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] \end{aligned}$$

Intuitively, this means the sigmoid is replaced with a lot of bell-shaped curves averaged together, one for every value of γ. Figure 2.1 shows how well different number of bells approximate σ.

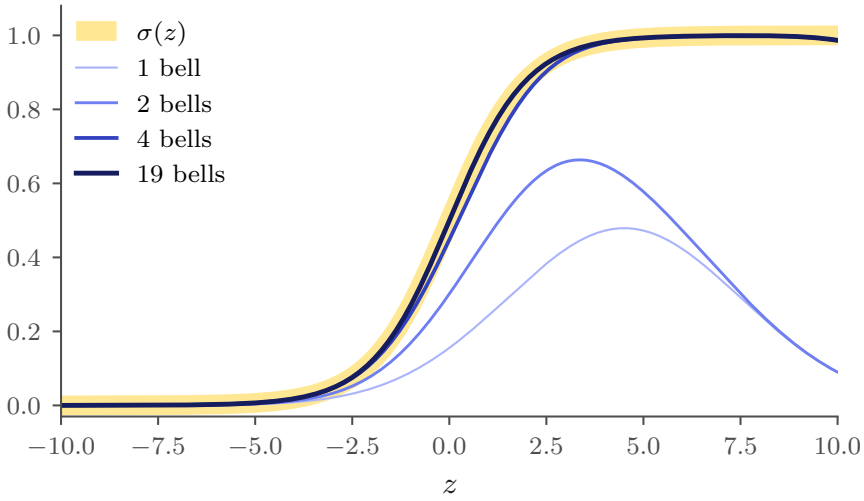


Figure 2.1.: The Pólya-Gamma identity graphically. The sigmoid represented as an average of bells $\sigma(z) = \mathbb{E}_\gamma\left[\frac{1}{2}e^{\frac{z}{2}-\frac{\gamma z^2}{2}}\right]$.

Now, we have a term without a sigmoid, but the x still appears in a quadratic form.

2.1.4. Trick 2: Hubbard-Stratonovich (HS)

To get rid of the quadratic form, we apply the Hubbard–Stratonovich identity [10, 11], that states:

$$\exp\left(-\tfrac{1}{2}ah^2\right) = \sqrt{\frac{1}{2\pi a}}\int_{-\infty}^{\infty}\exp\left(-\frac{u^2}{2a} - ihu\right)\mathrm{d}u, \tag{2.1}$$

Since $\frac{1}{\sqrt{2\pi a}} \exp\left(-\frac{u^2}{2a}\right)$ is the density of $\mathcal{N}(0, a)$, the integral is exactly the expectation of e^{-ihu} over $u \sim \mathcal{N}(0, a)$:

$$\exp\left(-\tfrac{1}{2}ah^2\right) = \mathbb{E}_{u\sim\mathcal{N}(0,\,a)}\left[e^{-ihu}\right]. \tag{2.2}$$

The trick makes a quadratic term linear at the cost of introducing a helper variable that we need to integrate over. In our term, x appears quadratically as well as linearly. To be able to apply HS, we complete the square (again using $z = w^T x$), where we write $c := \frac{1}{2\gamma}$:

$$\begin{aligned}
\frac{1}{2}z - \frac{1}{2}\gamma z^2 &= -\frac{1}{2}\gamma\left(z^2 - \frac{z}{\gamma}\right) \\
&= -\frac{1}{2}\gamma\left(z^2 - 2cz\right) \\
&= -\frac{1}{2}\gamma\left((z-c)^2 - c^2\right) \\
&= -\frac{1}{2}\gamma\,(z-c)^2 + \frac{1}{2}\gamma c^2 \\
&= -\frac{1}{2}\gamma\,(z-c)^2 + \frac{1}{8\gamma}.
\end{aligned} \tag{2.3}$$

Now we apply HS with $a = \gamma$ and $h = z - c$:

$$\begin{aligned}
\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right) &= \exp\left(\frac{1}{8\gamma}\right)\exp\left(-\frac{1}{2}\gamma\,(z-c)^2\right) \\
&= \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-iu\,(z-c)\right)\right]
\end{aligned} \tag{2.4}$$

Substituting back into $p(y \mid x^o)$, we get:

$$\begin{aligned}
p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] \\
&= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-iu\,(z-c)\right)\right]\right]
\end{aligned}$$

Intuitively, this means every bell is replaced with a lot of cosine waves $\cos(u(z-c))$ averaged together, one for every value of u (see Appendix A.1 for the derivation).

Figure 2.2 shows how well different numbers of waves approximate a bell.

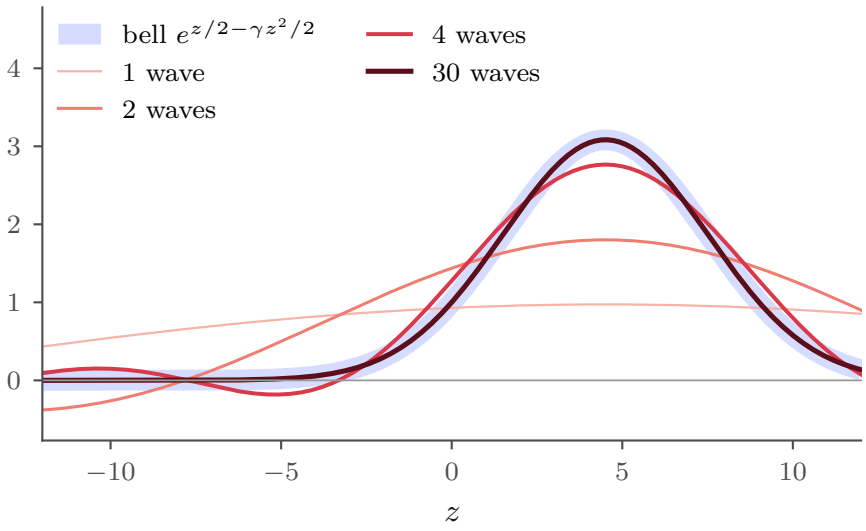


Figure 2.2.: The Hubbard-Stratonovich identity graphically. A bell from Figure 2.1 represented as an average of waves $e^{-\frac{\gamma}{2}(z-c)^2}$.

2.1.5. Arriving at the Characteristic Function

To continue, we want the expectation over x^m to be the innermost one, so we swap the order of the expectations:

$$p(y \mid x^o) = \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right]\right].$$

This is allowed because ...

The innermost expectation is now only over x^m, with x^o fixed, so we split $z = w^T x$ into its observed and missing parts:

$$z = w^T x = w_o^T x^o + w_m^T x^m.$$

The x^o part and c are constant under $\mathbb{E}_{p(x^m|x^o)}$ and factor out:

$$\begin{aligned}\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right] &= \mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(w_o^T x^o - c + w_m^T x^m\right)\right)\right] \\ &= \exp\left(iu\left(c - w_o^T x^o\right)\right)\cdot\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iw_m^T x^m u\right)\right]\end{aligned}$$

This remaining expectation is a known quantity: the characteristic function of a random variable X, $\phi_X(s) := \mathbb{E}\left[\exp(is^T X)\right]$. Writing $-iw_m^T x^m u = i(-uw_m)^T x^m$, we

get:

$$\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right] = \exp\left(iu\left(c - w_o^T x^o\right)\right) \cdot \phi_{x^m|x^o}(-uw_m).$$

Intuitively, the characteristic function gives the expectation of a wave under the distribution $p(z)$ of $z = w^T x$: for every frequency u, it gives the signed area under the wave times $p(z)$ (see Appendix A.2 for the derivation). Figure 2.3 shows this for two different waves, i.e. different values of u.

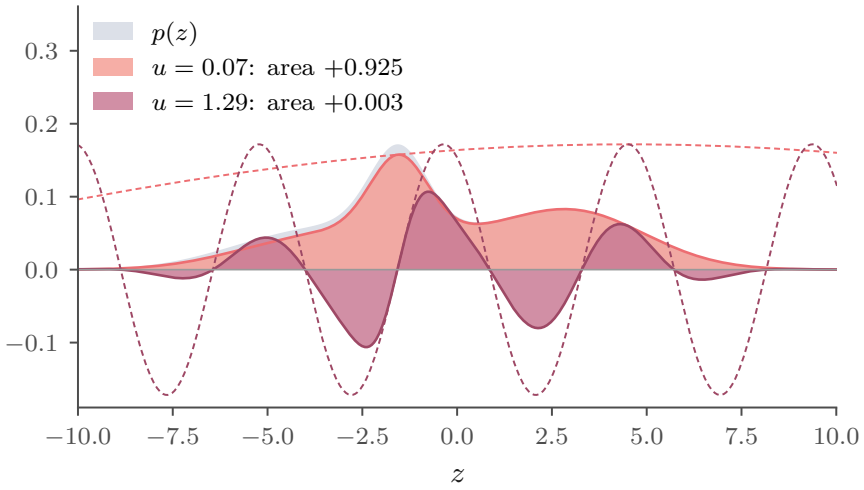


Figure 2.3.: The characteristic function graphically. The expectation of a wave under $p(z)$ is the signed area under the wave times $p(z)$ (shaded). Fast waves average to zero, while slower waves give a non-zero value.

2.1.6. Full Formula

Plugging the result of the previous subsection into $p(y \mid x^o)$ and writing out $c = \frac{1}{2\gamma}$, we obtain:

$$\begin{aligned} p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{p(x^m|x^o)}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(-iu\left(z-c\right)\right)\right]\right] \\ &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\exp\left(\frac{1}{8\gamma}\right)\exp\left(iu\left(\frac{1}{2\gamma} - w_o^T x^o\right)\right)\phi_{x^m|x^o}(-uw_m)\right] \end{aligned}$$

We propose this as a final expression that we can tractably evaluate.

2.1.7. Summary

The expected prediction $p(y \mid x^o)$ is the area under $\sigma(z)\, p(z)$. We replace the sigmoid function by a sum of weighted bells. We get the area under each weighted bell times $p(z)$, by replacing each bell with a sum of waves and put each wave frequency into the characteristic function, which gives the signed area under the wave times $p(z)$.

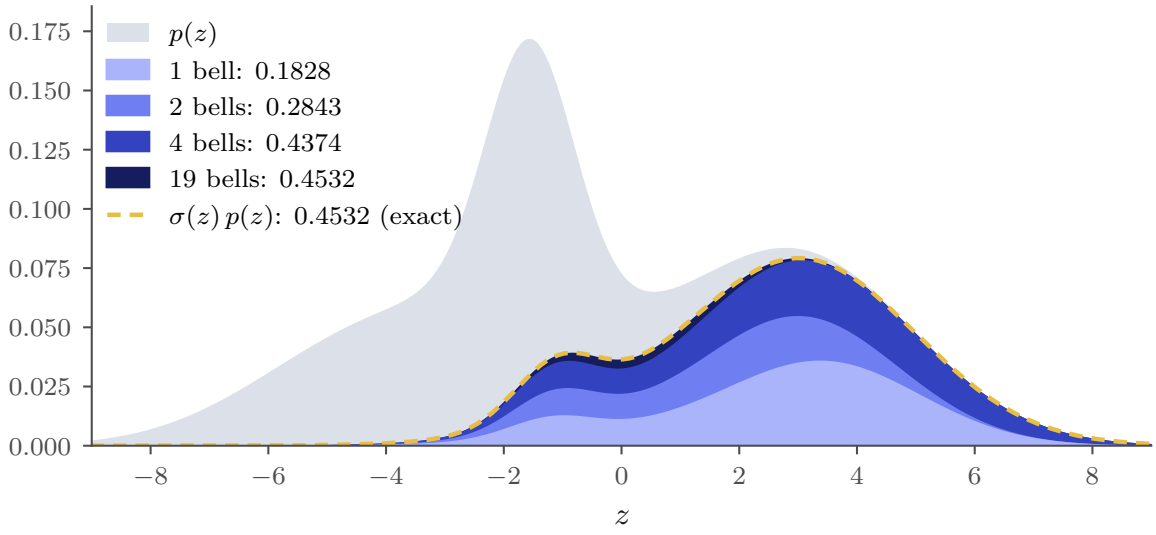


Figure 2.4.: The expected prediction as a sum of bells. $p(y \mid x^o)$ is the area under $\sigma(z)\, p(z)$ (dashed). Adding up the areas under the weighted bells times $p(z)$ (shaded), each computed from the characteristic function, recovers it.

2.2. Calculating the Two Expectations

After the two tricks, $p(y \mid x^o)$ is an expectation over two variables, γ and u. There is no closed form solution for this, so we need to approximate it. There are multiple ways to do this, but in all the principle is the same: we take a set of points (nodes) and associated weights, and replace the expectation with a weighted sum.

We write $f(u, \gamma)$ for the term inside the expectations,

$$f(u,\gamma) := \exp\left(\frac{1}{8\gamma}\right) \exp\left(iu\left(\frac{1}{2\gamma} - w_o^T x^o\right)\right) \phi_{x^m|x^o}(-uw_m).$$

2.2.1. Monte Carlo

The simplest way is to sample both expectations. We draw M samples $\gamma_j \sim \mathrm{PG}(1, 0)$, and for every γ_j we draw N samples $u_{ij} \sim \mathcal{N}(0, \gamma_j)$: Each sample is weighted equally

here, so the weights are $w_j = \frac{1}{M}$ and $\omega_{ij} = \frac{1}{N}$. Since they are not dependent on the sampled values, we can write them outside the sums. The resulting estimator is

$$p(y \mid x^o) \approx \frac{1}{2} \cdot \frac{1}{M} \sum_{j=1}^{M} \frac{1}{N} \sum_{i=1}^{N} f(u_{ij}, \gamma_j).$$

This is always unbiased, but the variance can be quite high and we need a large number of samples to get a good estimate.

2.2.2. Numerical Integration

Both expectations can also be computed by quadrature / numerical integration. The idea is, that instead of having to choose many points and weights randomly to get an acceptable error, we strategically choose a fixed set of specific points and weights that guarantees a certain level of accuracy.

Inner Expectation: Gauss–Hermite

The inner expectation is over $u \sim \mathcal{N}(0, \gamma)$. This is an exact case for a standard Gauss–Hermite quadrature (Section 1.7.2).

(Should this be explained in more detail?) We can easily get the nodes t_i and weights ω_i with `numpy.polynomial.hermite.hermgauss(N)`, which returns the N nodes and weights for the standard normal distribution $\mathcal{N}(0, 1)$. We then scale the nodes by $\sqrt{2\gamma}$ and the weights by $\frac{1}{\sqrt{\pi}}$ to transform them for our distribution.

$$\mathbb{E}_{u \sim \mathcal{N}(0,\gamma)}[f(u, \gamma)] \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^{N} \omega_i \, f\left(\sqrt{2\gamma}\, t_i, \gamma\right).$$

Plugging this into the full formula gives

$$p(y \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \mathbb{E}_{\gamma \sim \mathrm{PG}(1,0)} \left[\sum_{i=1}^{N} \omega_i \, f\left(\sqrt{2\gamma}\, t_i, \gamma\right) \right].$$

Outer Expectation: Gauss–PG

While for the inner expectation we can use a standard and widely known quadrature rule, there is no standard quadrature rule for an expectation over $\gamma \sim \mathrm{PG}(1, 0)$. So we need to construct one ourselves. We do this by using the Golub–Welsch algorithm

(Section 1.7.2), which is the default choice to construct quadrature nodes and weights for Gauss quadratures.

The Golub–Welsch algorithm only needs the moments of the distribution. For $\mathrm{PG}(1,0)$ they follow from its Laplace transform $\mathcal{L}(t) := \mathbb{E}[e^{-t\gamma}] = \operatorname{sech}(\sqrt{t/2})$, which is known in closed form. Then the k-th moment is, the k-th derivative at zero, up to its sign:

$$\mu_k := \mathbb{E}[\gamma^k] = (-1)^k \, \mathcal{L}^{(k)}(0).$$

We then apply the Golub–Welsch procedure and call the rule *Gauss–PG*. It is shown in Algorithm 1.

Algorithm 1 Gauss–PG quadrature rule

Input: number of nodes n

compute with 200 digits
for $k = 0, \ldots, 2n-1$ **do**
    $\mu_k \leftarrow (-1)^k \, \mathcal{L}^{(k)}(0)$ with $\mathcal{L}(t) = \operatorname{sech}(\sqrt{t/2})$
end for
$H_{ij} \leftarrow \mu_{i+j}$ and $H'_{ij} \leftarrow \mu_{i+j+1}$ for $i, j = 0, \ldots, n-1$
$R \leftarrow$ Cholesky factor of $H = R^T R$
$J \leftarrow R^{-T} H' R^{-1}$
$V \Lambda V^T \leftarrow$ eigendecomposition of J
$\gamma_j \leftarrow \lambda_j$ and $w_j \leftarrow v_{0j}^2$ for $j = 1, \ldots, n$
return $(\gamma_j, w_j)_{j=1}^n$

We do the computation with 200 digits, because the moments only differ very very slightly which makes the Cholesky decomposition fail. For regular floating point precision, the moments are already indistinguishable from about $n \geq 12$.

Since $\mathbb{E}[e^{-\gamma z^2/2}] = \mathcal{L}(z^2/2)$, a rule with n nodes (γ_j, w_j) approximates the sigmoid by a sum of bells,

$$\sigma(z) = \tfrac{1}{2} \, e^{z/2} \, \mathcal{L}\Big(\tfrac{z^2}{2}\Big) \approx \tfrac{1}{2} \, e^{z/2} \sum_{j=1}^{n} w_j \, e^{-\gamma_j z^2/2}. \tag{2.5}$$

Gauss–PG keeps its nodes away from $\gamma = 0$, so the factor $e^{1/(8\gamma_j)}$ in f stays small. Rules with nodes closer to zero can approximate the sigmoid *alone* more accurately, but fail with PCPG, becasue the factor blows up their error. These and other rules for the outer integral are described in Appendix A.3.

Combining the Gauss–PG nodes $(\gamma_j, w_j)_{j=1}^M$ with the Gauss–Hermite nodes $(t_i, \omega_i)_{i=1}^N$

yields the fully deterministic approximation

$$p(y \mid x^o) \approx \frac{1}{2\sqrt{\pi}} \sum_{j=1}^{M} w_j \sum_{i=1}^{N} \omega_i \, f\left(\sqrt{2\gamma_j}\, t_i, \gamma_j\right).$$

Sign Flip

Like it is somewhat visible in 2.1, for $z < 0$ the bells approximate the sigmoid quite well, even a small number of bells. For $z > 0$ it becomes harder and for larger z, i.e. $z > 10$, the sigmoid is very close to 1, whereas the sum of bells, no matter how many, tends towards 0 eventually. So we'd have a constant error of 1 for large z. More detail in (Appendix A.5).

To mitigate this we use that the sigmoid is point symmetric around $(0, \frac{1}{2})$:

$$\sigma(-z) = \frac{1}{1+e^z} = \frac{e^{-z}}{e^{-z}+1} = 1 - \sigma(z).$$

For $z > 0$ we can therefore compute $1 - \hat{\sigma}(-z)$ instead of $\hat{\sigma}(z)$. Both use the same nodes and the same approximation of $\mathcal{L}(z^2/2)$. The error at z becomes the error at $-z$. In practice this reduces the error over $|z| \leq 30$ of Gauss–PG with 20 nodes from about 1 to $6 \cdot 10^{-7}$.

The symmetry can be used for the whole expectation as well,

$$\mathbb{E}\left[\sigma(w^T x) \mid x^o\right] = 1 - \mathbb{E}\left[\sigma(-w^T x) \mid x^o\right],$$

which means the estimator can run with $-w$ and return one minus the result.

Since $z = w^T x$, with some x missing, is random, wether to flip is decided by the sign of the expected value $\mathbb{E}[w^T x \mid x^o] = w_o^T x^o + w_m^T \mathbb{E}[x^m \mid x^o]$. We do this for every point individually.

Since the expectation $\mathbb{E}[x^m \mid x^o]$ is available in closed form for GMMs and PCs, we can compute it efficiently and it doesn't add much additional cost.

2.3. The Characteristic Function of the Density Model

The only place where the density model is used in PCPG is the conditional characteristic function $\phi_{x^m|x^o}$. In this section we describe how to compute it for the two density models we use, GMMs and PCs.

2.3.1. Gaussian Mixture Models

For a Gaussian $X \sim \mathcal{N}(\mu, \Sigma)$, the characteristic function is known in closed form,

$$\phi_X(s) = \mathbb{E}\left[\exp(is^T X)\right] = \exp\left(is^T\mu - \tfrac{1}{2}s^T\Sigma s\right).$$

A GMM has the density $p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$. Since the expectation is linear, the characteristic function of a mixture is just a mixture of the characteristic functions of its components,

$$\phi_X(s) = \sum_{k=1}^{K} \pi_k \exp\left(is^T\mu_k - \tfrac{1}{2}s^T\Sigma_k s\right).$$

However, in PCPG we need the conditional characteristic function $\phi_{x^m|x^o}$, so the characteristic function of the missing features given the observed ones. To get it, we first need the conditional density $p(x^m \mid x^o)$, which with diagonal covariances, is the mixture:

$$p(x^m \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o)\, \mathcal{N}(x^m; \mu_k^m, \Sigma_k^m),$$

with the responsibilities

$$\tilde{\pi}_k(x^o) = p(k \mid x^o) = \frac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}.$$

Its components are the original ones restricted to the missing features, with mean μ_k^m and covariance Σ_k^m. The responsibilities are the posterior weights for each component. They state how well each component explains the observed features.

The conditional characteristic function is then just the characteristic function of the conditional mixture, so

$$\phi_{x^m|x^o}(s) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \exp\left(is^T\mu_k^m - \tfrac{1}{2}s^T\Sigma_k^m s\right).$$

We compute this for every point x^o once and reuse it for all frequencies s, since the responsibilities $\tilde{\pi}_k(x^o)$ only depend on the observed features. This is shown in algorithm 2.

Algorithm 2 Conditional characteristic function of a GMM

Input: GMM parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ with diagonal $\Sigma_k = \mathrm{diag}(\sigma_k^2)$, partial observation x^o, frequencies $s_1, \ldots, s_L$ over the missing features

1: **for** $k = 1, \ldots, K$ **do**
2: $\quad \tilde{\pi}_k \leftarrow \dfrac{\pi_k \mathcal{N}(x^o; \mu_k^o, \Sigma_k^o)}{\sum_j \pi_j \mathcal{N}(x^o; \mu_j^o, \Sigma_j^o)}$
3: **end for**
4: **for** $l = 1, \ldots, L$ **do**
5: $\quad \phi_l \leftarrow \sum_{k=1}^K \tilde{\pi}_k \exp\left(i s_l^T \mu_k^m - \frac{1}{2} \sum_{d \in \mathrm{mis}} s_{ld}^2 \sigma_{kd}^2\right)$
6: **end for**
7: **return** $\phi_1, \ldots, \phi_L$

2.3.2. Probabilistic Circuits

How to compute the CF of a general PC.

PC frameworks usually don't provide a method for it. So we need to implement it ourselves.

The framework SPFlow already offers conditional sampling so we would only need to implement the CF. The framework cirkit doesn't have either.

The PC would also need to be able to give the means of z for the flip.

We could also implement a very simple PC from scratch?

2.4. PCPG Algorithm

Putting everything together leads to the PCPG algorithm shown in Algorithm 3. For one test point, it decides the sign flip, evaluates the density's conditional characteristic function at the frequencies given by the nodes, and returns the weighted sum of f. The Gauss–Hermite and Gauss–PG nodes depend neither on the point nor on the classifier or the density model, so they are computed once beforehand. The density model is only used through its characteristic function, so the algorithm works for every model that can expose one.

Algorithm 3 PCPG

Input: LR weights w, partial observation x^o, density model with conditional characteristic function $\phi_{x^m|x^o}$, Gauss–PG nodes and weights $(\gamma_j, w_j)_{j=1}^M$, Gauss–Hermite nodes and weights $(t_i, \omega_i)_{i=1}^N$

Split $w = (w_o, w_m)$ by the observed/missing mask
flip $\leftarrow \mathbb{E}[w^T x \mid x^o] > 0$
if flip **then**
    $w_o \leftarrow -w_o, \quad w_m \leftarrow -w_m$
end if
$u_{ij} \leftarrow \sqrt{2\gamma_j}\, t_i$ for all i, j
$\phi_{ij} \leftarrow \phi_{x^m|x^o}(-u_{ij}\, w_m)$ for all i, j
$S \leftarrow \sum_{j=1}^M \sum_{i=1}^N w_j\, \omega_i \exp\left(\frac{1}{8\gamma_j}\right) \exp\left(iu_{ij}\left(\frac{1}{2\gamma_j} - w_o^T x^o\right)\right) \phi_{ij}$
$p \leftarrow \frac{1}{2\sqrt{\pi}} \operatorname{Re}(S)$
return $1 - p$ if flip, else p

One can come up with other variants of PCPG by changing how the two expectations are computed. The inner, the outer, or both could be sampled instead. For both also different quadrature rules could be used. Appendix A.4 lists different combinations. In the rest of this thesis, *PCPG* refers to Algorithm 3, and *PCPG-MC* to the variant that samples both expectations.

2.5. Training with PCPG

Up to now, the classifier w was trained on complete data and features were only missing at test time. However, in practice, training data could also have missing features.

In this section we show how PCPG can also be used in training.

2.5.1. Fitting the density

The density model $p(x)$ is fit first. Here PCPG is not needed, and in general this is a standard and well-established procedure. The density is fit, by maximizing the marginal likelihood of the observed features x^o:

$$p(x^o) = \int p(x = (x^o, x^m))\, \mathrm{d}x^m.$$

In general this integral is hard, but can be computed tractably for a GMM or a PC.

For a GMM, we can see this one component k at a time. A component is a Gaussian $\mathcal{N}(\mu_k, \Sigma_k)$, and since our GMM uses diagonal covariances $\Sigma_k = \text{diag}(\sigma_{k1}^2, \ldots, \sigma_{kD}^2)$, every feature d, within a component, has its own one-dimensional Gaussian with mean μ_{kd} and variance σ_{kd}^2. This means that the features are independent of each other within a component. Since the density of independent variables is the product of their individual densities, the density of a point x in component k can be written as

$$p(x \mid k) = \prod_{d=1}^{D} \mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2).$$

Now when a feature d is missing, integrating over x_d only touches its own term $\mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2)$. Because that term is a density, it integrates to one and it can simply be dropped out of the product. Then only the terms of the observed features remain,

$$p(x^o) = \sum_{k=1}^{K} \pi_k \prod_{d \in \text{obs}} \mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2).$$

A PC works the same way. Its leaves are the one-dimensional densities of single features, and when the circuit is smooth and decomposable, the integral over a missing feature can be moved down to its leaves. There the leaves of the missing features are simply set to one and the circuit is evaluated as usual.

$p(x)$ is fit once and kept fixed for the remainder of the training process.

2.5.2. Fitting the classifier

When we already have a way to predict with missing data at test time, training the classifier with missing data is actually quite simple. We just use the same prediction method, that we use at test time, namely the expected prediction over all plausible values of the missing features:

$$p_n = p(y_n = 1 \mid x_n^o) = \mathbb{E}_{p(x^m \mid x_n^o)} \left[\sigma(w^T x) \right].$$

As in an standard logistic regression, w should make true labels as likely as possible. This means we want to maximize the likelihood of true labels, i.e. the probability the classifier assigns to them, or equivalently, minimize the negative log-likelihood. This leads to the following loss function:

$$J(w) = -\frac{1}{n_{\text{train}}} \sum_{n=1}^{n_{\text{train}}} \log p(y_n \mid x_n^o) + \frac{1}{2Cn_{\text{train}}} \|w\|^2. \tag{2.6}$$

Here $p(y_n \mid x_n^o)$ means the probability of the true label of point n: it is p_n if $y_n = 1$ and $1 - p_n$ if $y_n = 0$:

$$\log p(y_n \mid x_n^o) = y_n \log p_n + (1 - y_n) \log(1 - p_n).$$

This is the familiar binary cross-entropy.

The second term here is the L_2 penalty of scikit-learn's logistic regression, with $C = 1$. It keeps w from growing without bound and makes the optimum unique.

As mentioned we compute the expected prediction $p(y_n \mid x_n^o)$ with PCPG, and use automatic differentiation to compute the gradient $\nabla_w J(w)$ of the loss function. Then we can use any gradient-based optimizer, like Adam, and find the optimal w.

2.5.3. Algorithm

Algorithm 4 puts both stages together. First, we fit the density model $p_\theta(x)$, then the classifier starts at $w = 0$ and is trained by Adam on minibatches. Since the expected predictions of PCPG can lie slightly outside $[0, 1]$ due to the quadrature error, we clamp them to $[\varepsilon, 1 - \varepsilon]$.

Algorithm 4 Training a logistic regression with PCPG

Input: incomplete training points $(x_n^o, y_n)_{n=1}^{n_{\text{train}}}$, PCPG nodes and weights, penalty C, epochs E, batch size B, learning rate η, ε

Fit the density model: $\theta \leftarrow \arg\max_\theta \sum_n \log p_\theta(x_n^o)$
$w \leftarrow 0$
for $e = 1, \ldots, E$ **do**
    for every minibatch $\mathcal{B}$ of B training points, in random order **do**
        for $n \in \mathcal{B}$ **do**
            $p_n \leftarrow \text{PCPG}(w, x_n^o)$ with Algorithm 3, clamped to $[\varepsilon, 1 - \varepsilon]$
        end for
        $J \leftarrow -\frac{1}{B} \sum_{n \in \mathcal{B}} (y_n \log p_n + (1 - y_n) \log(1 - p_n)) + \frac{1}{2Cn_{\text{train}}} \|w\|^2$
        $g \leftarrow \nabla_w J$
        $w \leftarrow \text{Adam}(w, g, \eta)$
    end for
end for
return w and p_θ

2.6. A Direct Route for Gaussian Mixtures

Although we use GMMs as density models for PCPG throughout, for a GMM the two tricks are actually not needed: the expected prediction can be computed directly. We include them anyway, to demonstrate the general approach in a simpler setting and to be able to compare the results of PCPG with a reference.

In this section we show how the expected prediction can be computed directly for a GMM.

2.6.1. Reducing the Integral to One Dimension

The prediction $\sigma(w^T x)$ does't depend on individual features, but only on the single number

$$z = w^T x = w_o^T x^o + w_m^T x^m.$$

The observed part $w_o^T x^o$ is fixed. Only the missing part is random. So instead of averaging σ over all missing features, we can average it just over z, and that is always a one-dimensional number. This reduces the dimensionality of the integral from the number of missing features to one, which makes it much easier to compute. The only thing we need for this is the distribution $p(z \mid x^o)$.

2.6.2. The Distribution of z

For a GMM, the distribution of z is relatively easy to find.

Let's first rewrite z as,

$$z = w_o^T x^o + \sum_{d \in \text{mis}} w_d \, x_d,$$

where the only uncertain part is the sum of the missing features.

Then recall that every missing feature is a one-dimensional Gaussian of its own, and the missing features are independent of each other,

$$x_d \mid k \sim \mathcal{N}(\mu_{kd}, \sigma_{kd}^2) \quad \text{for every } d \in \text{mis}.$$

A weighted sum of independent Gaussians is again Gaussian. So within component k, z is Gaussian as well, $z \mid k \sim \mathcal{N}(m_k, v_k)$, and we only need its mean m_k and its variance v_k.

The mean of a sum is the sum of the means, and the mean of $w_d x_d$ is $w_d \mu_{kd}$. So the mean of z is z with every missing feature replaced by its mean,

$$m_k = \mathbb{E}[z \mid k] = c + \sum_{d \in \text{mis}} w_d \, \mathbb{E}[x_d \mid k] = w_o^T x^o + \sum_{d \in \text{mis}} w_d \, \mu_{kd}.$$

Similarly, the variance of their sum is the sum of their variances, and multiplying x_d by w_d multiplies its variance by w_d^2. So

$$v_k = \text{Var}[z \mid k] = \sum_{d \in \text{mis}} \text{Var}[w_d \, x_d \mid k] = \sum_{d \in \text{mis}} w_d^2 \, \sigma_{kd}^2.$$

Lastly, we don't know which component the point comes from. Every component gives its own Gaussian for z, and the observed features tell us how likely each component is, through the responsibilities $\tilde{\pi}_k(x^o)$ of Section 2.3.1. The distribution of z is therefore the mixture of these Gaussians, weighted by the responsibilities,

$$p(z \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \mathcal{N}(z; m_k, v_k).$$

2.6.3. Computing the Expected Prediction

The expected prediction then splits into one average per component,

$$p(y \mid x^o) = \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \, \mathbb{E}_{z \sim \mathcal{N}(m_k, v_k)} \left[\sigma(z) \right].$$

Each of these averages is a one-dimensional integral of the sigmoid against a Gaussian. While it doesn't have a closed form solution, it can be very accurately computed using numerical integration, like Gauss-Hermite quadrature (Section 2.2.2) or adaptive quadrature using `scipy.integrate.quad`.

$$p(y \mid x^o) \approx \frac{1}{\sqrt{\pi}} \sum_{k=1}^{K} \tilde{\pi}_k(x^o) \sum_{i=1}^{N} \omega_i \, \sigma\left(m_k + \sqrt{2 v_k} \, t_i \right).$$

2.6.4. The Inner Expectation in Closed Form

Need to rewrite.

The mixture $p(z \mid x^o)$ can also be used inside PCPG. After the first trick and swapping the two expectations, the expected prediction is

$$p(y \mid x^o) = \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\mathbb{E}_{z\sim p(z|x^o)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right].$$

PCPG computes the inner expectation over z with the second trick and Gauss–Hermite quadrature. For a GMM it can be computed exactly instead, because the bell $\exp(\frac{1}{2}z - \frac{1}{2}\gamma z^2)$ times a Gaussian density is again a Gaussian, up to a constant.

Within component k we have $z \sim \mathcal{N}(m_k, v_k)$, and we drop the index k for the derivation. Writing out the density, the expectation is

$$\mathbb{E}_{z\sim\mathcal{N}(m,v)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] = \frac{1}{\sqrt{2\pi v}}\int_{-\infty}^{\infty}\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2 - \frac{(z-m)^2}{2v}\right)\mathrm{d}z.$$

The exponent is quadratic in z. We collect its terms by powers of z and write $a := \gamma + \frac{1}{v}$ and $b := \frac{1}{2} + \frac{m}{v}$:

$$\begin{aligned}\frac{1}{2}z - \frac{1}{2}\gamma z^2 - \frac{(z-m)^2}{2v} &= -\frac{1}{2}\left(\gamma + \frac{1}{v}\right)z^2 + \left(\frac{1}{2} + \frac{m}{v}\right)z - \frac{m^2}{2v} \\ &= -\frac{1}{2}az^2 + bz - \frac{m^2}{2v} \\ &= -\frac{1}{2}a\left(z - \frac{b}{a}\right)^2 + \frac{b^2}{2a} - \frac{m^2}{2v}.\end{aligned}$$

The last step completes the square, as in the second trick. The last two terms do not depend on z and can be taken out of the integral. What remains is the integral of an unnormalised Gaussian with variance $\frac{1}{a}$, which is $\sqrt{2\pi/a}$. Therefore

$$\begin{aligned}\mathbb{E}_{z\sim\mathcal{N}(m,v)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] &= \frac{1}{\sqrt{2\pi v}}\sqrt{\frac{2\pi}{a}}\exp\left(\frac{b^2}{2a} - \frac{m^2}{2v}\right) \\ &= \frac{1}{\sqrt{va}}\exp\left(\frac{b^2}{2a} - \frac{m^2}{2v}\right).\end{aligned}$$

Both parts simplify. In front, $va = \gamma v + 1$. In the exponent, we use $b = \frac{v+2m}{2v}$ and

$a = \frac{1+\gamma v}{v}$:

$$\begin{aligned}\frac{b^2}{2a} - \frac{m^2}{2v} &= \frac{(v+2m)^2}{8v(1+\gamma v)} - \frac{m^2}{2v} \\ &= \frac{(v+2m)^2 - 4m^2(1+\gamma v)}{8v(1+\gamma v)} \\ &= \frac{v^2 + 4vm - 4\gamma v m^2}{8v(1+\gamma v)} \\ &= \frac{v + 4m - 4\gamma m^2}{8(1+\gamma v)}.\end{aligned}$$

With the index k back, the inner expectation of component k is

$$g_k(\gamma) := \mathbb{E}_{z\sim\mathcal{N}(m_k,v_k)}\left[\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right)\right] = \frac{1}{\sqrt{1+\gamma v_k}}\exp\left(\frac{v_k + 4m_k - 4\gamma m_k^2}{8(1+\gamma v_k)}\right).$$

As a check, for $\gamma = 0$ this gives $\exp(\frac{1}{2}m_k + \frac{1}{8}v_k)$, which is the known mean $\mathbb{E}[e^{z/2}]$ of a log-normal variable.

The inner expectation over the whole mixture is the weighted sum of the components, and only the outer expectation over γ is left. Computing it with the Gauss–PG nodes and weights $(\gamma_j, w_j)_{j=1}^M$ of Section 2.2.2 gives

$$\begin{aligned}p(y \mid x^o) &= \frac{1}{2}\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}\left[\sum_{k=1}^K \tilde{\pi}_k(x^o)\, g_k(\gamma)\right] \\ &\approx \frac{1}{2}\sum_{j=1}^M w_j \sum_{k=1}^K \tilde{\pi}_k(x^o)\, g_k(\gamma_j).\end{aligned}$$

The sign flip of Section 2.2.2 carries over unchanged: replacing w by $-w$ only changes the sign of every m_k, and leaves v_k as it is.

As an estimator this is not useful on its own, since the one-dimensional integrals of the previous subsection are simpler and exact. But it is PCPG with a perfect inner quadrature: the only error left is the error of the Gauss–PG rule. Comparing it with PCPG therefore separates the error of the outer quadrature from the error of the inner one.

2.6.5. Why This Does Not Work for General PCs

The direct route needs two things: that we can go through the components one by one, and that z is Gaussian within each of them. A PC is also a mixture, but the

number of its components grows exponentially with its size, so going through them one by one is only possible for small circuits.

Also a general PC could have non-Gaussian leaves, which would make z not Gaussian within a component.

3. Experimental Results

This chapter presents the experiments: first how accurately PCPG estimates the expected prediction, then how well a logistic regression can be trained with PCPG on incomplete data. Discussion of the results follow in Chapter 4.

3.1. Estimating the Expected Prediction

Setup. The data are generated with scikit-learn's `make_classification`: 10,000 samples, of which half of the features are informative and the other half are independent noise; 8,000 samples are used for training and 2,000 for testing. The density $p(x)$ is a GMM with $K = 4$ diagonal components, fit on the complete training points, and the classifier is a standard logistic regression from scikit-learn. The number of features goes from 5 to 5,000, each one being a factor $\sqrt{10}$ apart. For every test point a fraction $\rho \in \{0.3, 0.5, 0.8\}$ of the features is hidden at random.

MC and PCPG get the same budget of 1,200 evaluations: MC draws 1,200 samples, and PCPG uses $M = 20$ Gauss–PG nodes and $N = 60$ Gauss–Hermite nodes. PCPG exact inner is PCPG with the inner expectation in closed form (Section 2.6.4), so its error is that of the Gauss–PG rule alone. The error is the RMSE over 50 test points against the exact value, which we compute from the Gaussian mixture of $z \mid x^o$ (Section 2.6) with adaptive quadrature. The time is that of one batched call over all 2,000 test points on a GPU.

We run the comparison in two variants, which differ only in terms of standardisation and regularisation: one for the raw data with default regularisation $C = 1$, and one for the data with standardised features and stronger regularisation $C = 10^{-3}$. The standardisation is done on the training points, and the same transformation is applied to the test points. Some additional variants are in Appendix A.6.

Raw data, default classifier. We see that with only few features PCPG clearly outperforms MC by some orders of magnitude. However after about 1000 features, the error of PCPG grows rapidly and even gets larger than that of MC. Theories about why this happens are discussed in detail in Section 4.1.2, but the main reason

is that the standard deviation of z grows with the number of features, which makes the quadrature rules much less accurate.

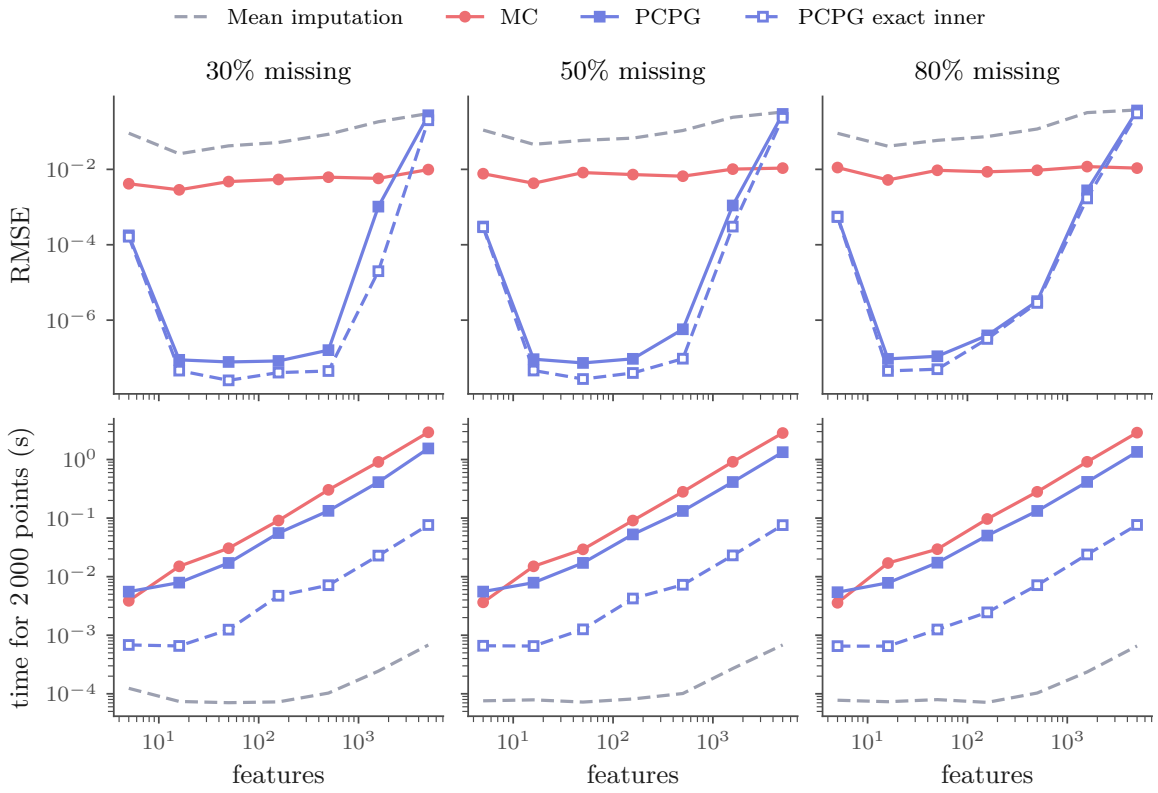


Figure 3.1.: Raw data, logistic regression with $C = 1$.

Standardise data, stronger regularisation. The first idea is to standardise the features to mean 0 and variance 1 on the training points, and apply the same transformation to the test points. This helps but does not completely solve the problem. One also has to ensure that the magnitude of w is controlled. The best way we found to do this, without changing the classifier too much, is to use different regularisation strengths C for the logistic regression. In (Table 3.1) we show different values of C and the resulting standard deviation of z and test accuracy.

Table 3.1.: The logistic regression on the standardised data for different regularisations C: the standard deviation of $z = w^T x$ over the training points and the accuracy on the test points.

	$C = 1$		$C = 10^{-2}$		$C = 10^{-3}$	
d	sd(z)	**test acc.**	sd(z)	**test acc.**	sd(z)	**test acc.**
5	3.70	0.888	2.50	0.892	1.12	0.888
16	1.40	0.721	1.27	0.721	0.78	0.713
50	2.14	0.797	1.80	0.797	0.99	0.794
158	2.64	0.805	2.14	0.805	1.16	0.798
500	3.07	0.813	2.36	0.810	1.25	0.806
1581	5.43	0.764	2.88	0.777	1.36	0.776
5000	12.42	0.702	3.79	0.713	1.65	0.722

We see, that the stronger regularisation costs no accuracy: from $d = 1581$ on, the test accuracy is even higher than with $C = 1$, which overfits there.

Then redoing the first experiment with standardised features and $C = 10^{-3}$, yields (Figure 3.2), and PCPG reaches an RMSE of about 10^{-7} in every setting, clearly outperforming MC everywhere. This shows, that the error of PCPG depends mainly on the spread of z, not on the number of features. Whenever z stays on a moderate scale, PCPG is accurate to about 10^{-7} for every d and missing rate.

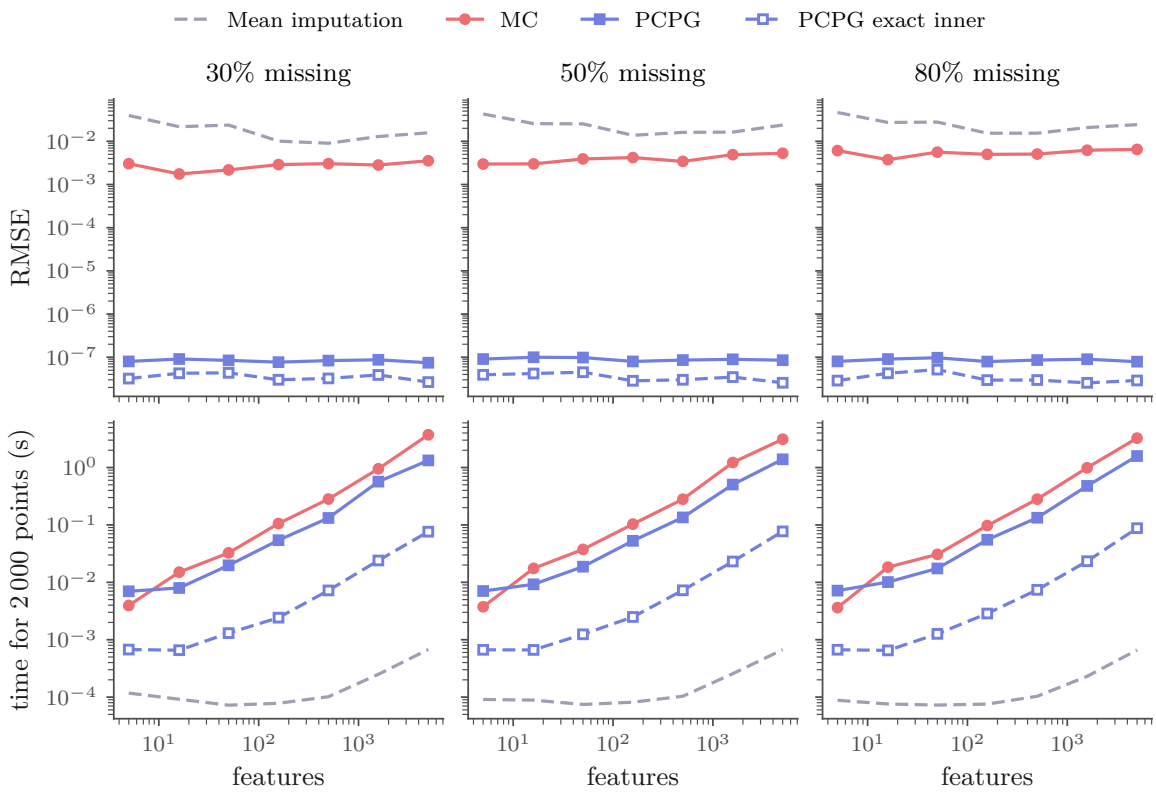


Figure 3.2.: Standardised features, logistic regression with $C = 10^{-3}$.

Runtime. The runtime for MC and PCPG it grows linearly in d, and from $d = 16$ on, PCPG is about twice as fast as MC at the same budget. PCPG exact inner needs no characteristic function and takes less than 0.1 s. Mean Imputation is of course the fastest.

3.2. Training with Missing Features

In this section we train a logistic regression on incomplete data with PCPG inside the loss (Section 2.5) and compare it against other typical ways of training on incomplete data.

3.2.1. Compared Models

All models are again standard logistic regressions without intercept. They differ only in the training points they see and in the prediction their loss is built from (Table 3.2):

- **Complete data:** an ordinary logistic regression from sklearn on the complete training points. It acts as an oracle, the classifier that training could reach if no features were missing.
- **Mean imputation:** the same ordinary logistic regression on the training points with every missing feature replaced by the mean of its observed values.
- **MC, PCPG, Exact:** the expected marginal likelihood (2.6) on the incomplete training points, estimated by Monte Carlo, deterministic PCPG, or computed exactly. Exact uses the direct route for Gaussian mixtures (Section 2.6) with Gauss–Hermite quadrature (a slightly higher error than adaptive quad, but much faster) for every component.

Model	**Training points**	**Prediction in the loss**
Complete data	complete x	$\sigma(w^T x)$
Mean imputation	x^o, x^m filled by means	$\sigma(w^T x)$
MC	x^o	$\frac{1}{S}\sum_s \sigma(w^T x^{(s)})$
PCPG	x^o	PCPG, $M \times N$ nodes
Exact	x^o	$\sum_k \tilde{\pi}_k(x^o)\mathbb{E}_{\mathcal{N}(m_k,v_k)}[\sigma(z)]$

Table 3.2.: The compared models. The last three minimise the marginal likelihood (2.6), the first two an ordinary logistic regression loss. All use the same L_2 penalty.

MC, PCPG and Exact are all trained with Adam on the same minibatches, starting from $w = 0$, so that they differ only in the estimate of the expectation.

3.2.2. Setup

The data for the experiment is setup exactly as in Section 3.1. The density is also again a GMM with $K = 4$ diagonal components, now fit on the incomplete training points. PCPG uses $M = 20$ Gauss–PG nodes and $N = 60$ Gauss–Hermite nodes, and

MC gets the same budget of $S = 1{,}200$ samples per point and step. Exact uses 64 Gauss–Hermite nodes per component. Adam runs for 100 epochs with minibatches of 1,024 points and a learning rate of 0.01.

We train one configuration with the raw data and default regularisation $C = 1$, and one with the standardised data and stronger regularisation $C = 10^{-3}$.

3.2.3. Evaluation

The evaluation tries to answer two questions.

- **How good is the training procedure?** MC, PCPG and Exact are trained identically. Any difference between their classifiers comes from how they estimate the expectation. We measure the mean absolute difference between the expected predictions of a classifier and those of the Exact classifier on the incomplete test points, where in the evaluation all are predicted exactly.
- **How good is the final classifier?** We want to see if the accuracy of $\sigma(w^T x)$ is on par with different training methods. We compute the prediction accuracy once with complete test points and once with partly missing test points. For the incomplete test points, every model uses it's "own" rule: mean imputation fill the missing features with their training means, MC, PCPG and Exact use their own estimate of the expected prediction and complete (training) data also uses the exact estimate.

3.2.4. Results

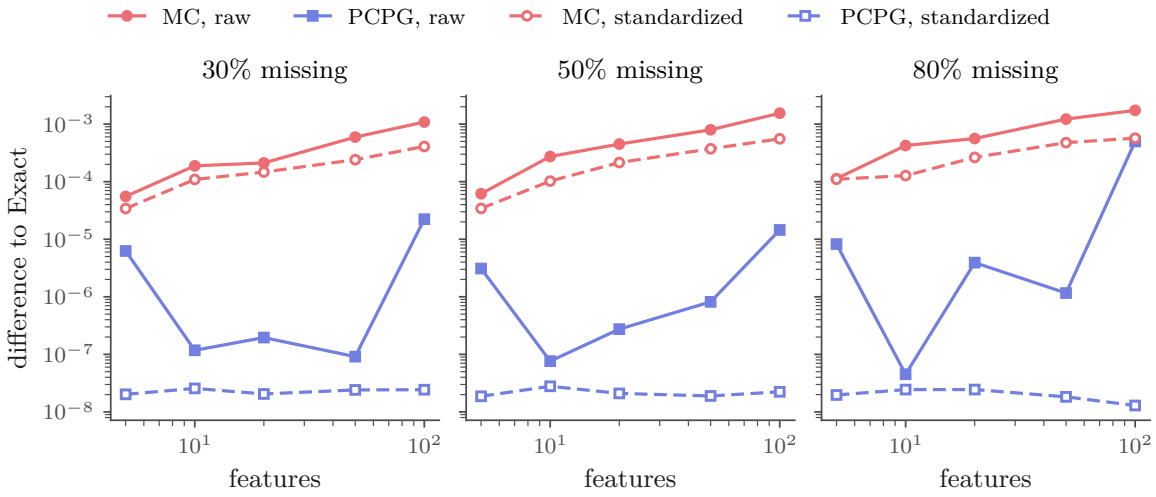


Figure 3.3.: Mean absolute difference between the expected predictions of the classifiers trained with MC and PCPG and those of the classifier trained with the exact expectation, on the incomplete test points, against the number of features d. Solid: raw data with $C = 1$, dashed: standardised data with $C = 10^{-3}$.

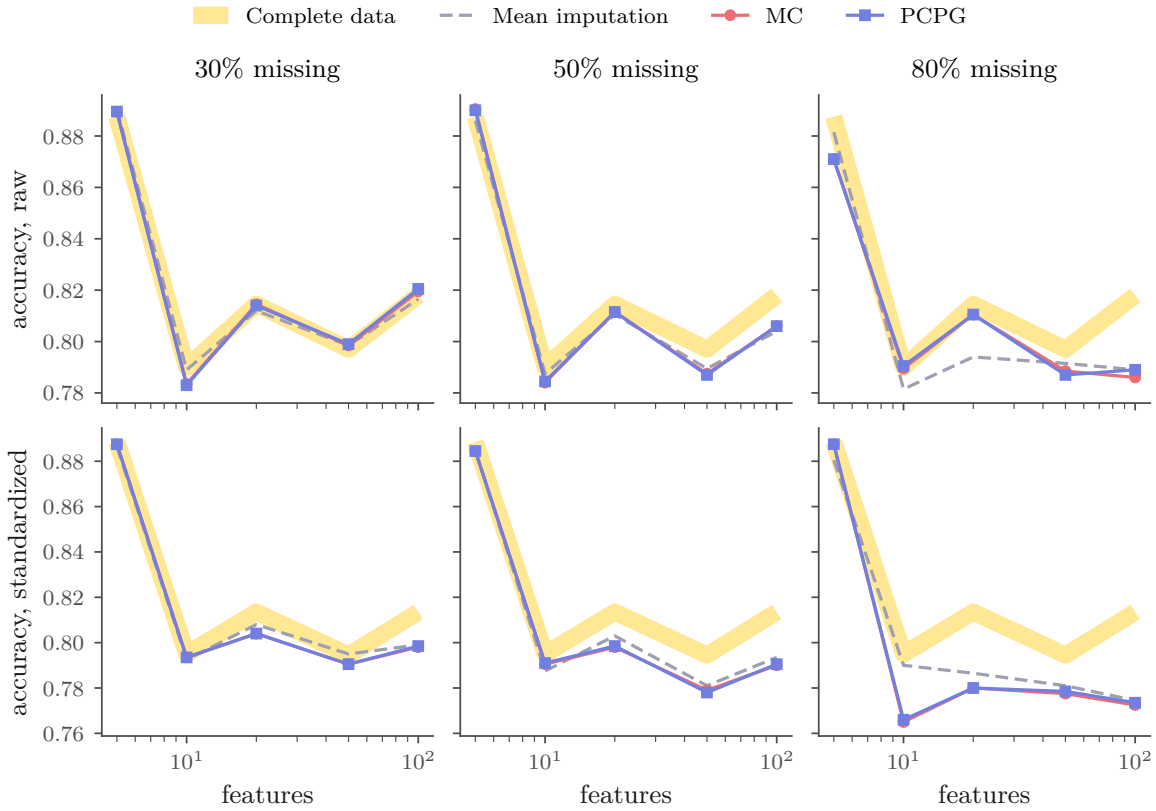


Figure 3.4.: Accuracy of $\sigma(w^T x)$ on the complete test points against the number of features d. Top: raw data with $C = 1$, bottom: standardised data with $C = 10^{-3}$. The target is the classifier trained on complete data (yellow band).

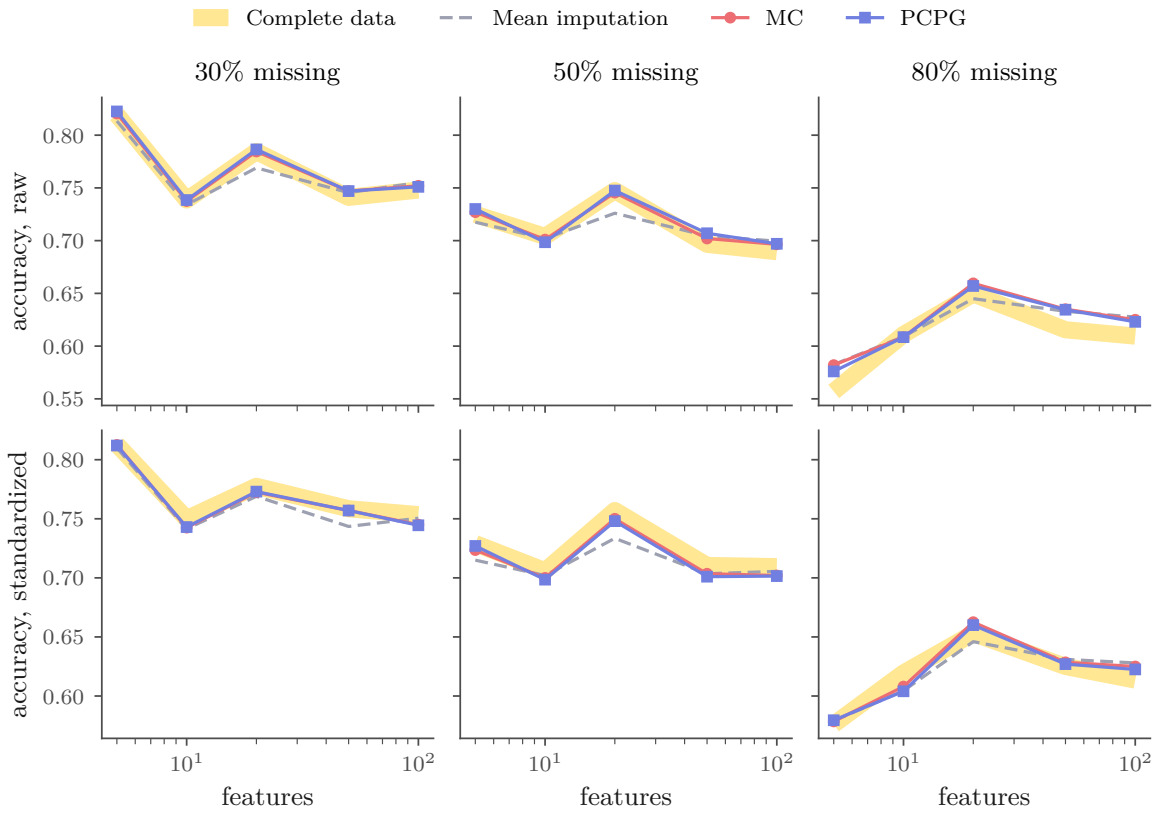


Figure 3.5.: Accuracy on the incomplete test points, every model with its own prediction rule: mean imputation for mean imputation, the estimated expected prediction for MC and PCPG. Top: raw data, bottom: standardised data. The target is the classifier trained on complete data with the exact expected prediction (yellow band).

4. Discussion

4.1. Predicting with PCPG

4.1.1. Bias and Variance

We compare the bias and variance of MC with PCPG-MC, which samples both expectations, and PCPG, which computes both by quadrature.

MC Estimator

The MC estimator draws N samples $x^{m(i)} \sim p(x^m \mid x^o)$ and returns

$$\widehat{p}_{\mathrm{MC}}(N) = \frac{1}{N} \sum_{i=1}^{N} \sigma\Big(w^T x^{(i)}\Big) .$$

Bias. MC is unbiased for all $N \geq 1$: sampling from $p(x^m \mid x^o)$ and averaging $\sigma(w^T x)$ is a direct Monte Carlo estimate of the target expectation, so $\mathbb{E}[\widehat{p}_{\mathrm{MC}}(N)] = p(y \mid x^o)$ exactly.

Variance. The draws are i.i.d., so the variance of their average is the per-sample variance $\sigma^2_{\mathrm{MC}} := \mathrm{Var}_{p(x^m|x^o)}\Big[\sigma(w^T x)\Big]$ divided by N. Since $\sigma(\cdot) \in (0,1)$, the per-sample variance is bounded: for any $[0,1]$-valued random variable Y,

$$\mathrm{Var}(Y) = \mathbb{E}[Y^2] - \mathbb{E}[Y]^2 \leq \mathbb{E}[Y] - \mathbb{E}[Y]^2 = \mathbb{E}[Y](1 - \mathbb{E}[Y]) \leq \tfrac{1}{4},$$

using $Y^2 \leq Y$ on $[0,1]$ and $p(1-p) \leq 1/4$. Therefore

$$\mathrm{Var}[\widehat{p}_{\mathrm{MC}}(N)] = \frac{\sigma^2_{\mathrm{MC}}}{N} \leq \frac{1}{4N}.$$

PCPG-MC Estimator

PCPG-MC draws M outer samples $\gamma_j \sim \mathrm{PG}(1,0)$ and, for each of them, L inner samples $u_{ij} \sim \mathcal{N}(0,\gamma_j)$ (Section 2.2.1). With $Z := \frac{1}{2}\mathrm{Re}[f(u,\gamma)]$ for a single sample, it returns

$$\widehat{p}_{\text{PCPG-MC}}(M,L) = \frac{1}{ML}\sum_{j=1}^{M}\sum_{i=1}^{L} Z_{ij}.$$

Bias. PCPG-MC is unbiased as well. Both expectations are approximated by ordinary MC, so by linearity of expectation the estimator equals the true double integral, which is $p(y \mid x^o)$ by the PG and Hubbard–Stratonovich identities.

Output range. Since $|\phi_{x^m|x^o}| \leq 1$, a single sample satisfies $|Z| \leq \frac{1}{2}e^{1/(8\gamma)}$. Because $\gamma \sim \mathrm{PG}(1,0)$ can be arbitrarily small, this bound is unbounded, so individual PCPG samples are not confined to $[0,1]$ and can be negative or exceed 1, while every MC sample lies in $(0,1)$.

Variance. The estimator has two nested levels of randomness. For a fixed γ_j, the average of its L inner samples has the conditional variance $\sigma^2_{A,\gamma_j}/L$, with $\sigma^2_{A,\gamma} := \mathrm{Var}_{u|\gamma}[Z \mid \gamma]$. The law of total variance over both levels and the M independent outer samples give

$$\mathrm{Var}[\widehat{p}_{\text{PCPG-MC}}(M,L)] = \frac{\sigma_A^2}{ML} + \frac{\sigma_B^2}{M}, \qquad \sigma_A^2 := \mathbb{E}_\gamma\left[\sigma^2_{A,\gamma}\right], \quad \sigma_B^2 := \mathrm{Var}_\gamma\left[\mathbb{E}_{u|\gamma}[Z \mid \gamma]\right].$$

σ_A^2 measures how much the inner expectation fluctuates for a typical γ, and σ_B^2 how much γ itself shifts the result.

The inner variance is infinite. The problem comes from completing the square in Trick 2, which introduced the factor $e^{1/(8\gamma)}$. For a fixed z, the inner expectation is known exactly from the HS identity,

$$\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[e^{-iu(z-c)}\right] = \exp\left(-\tfrac{1}{2}\gamma\,(z-c)^2\right) \approx e^{-1/(8\gamma)} \quad \text{for small } \gamma,$$

which is tiny. Multiplied by $e^{1/(8\gamma)}$ it gives a value of order one. So the inner average has to produce a tiny number from terms of modulus one that almost cancel, and every bit of noise in that average is amplified by $e^{1/(8\gamma)}$, which is already about $2.7\cdot 10^5$ for $\gamma = 0.01$. In terms of variance, a single inner sample has the second moment $e^{1/(4\gamma)}$. Near zero the density of $\mathrm{PG}(1,0)$ decays only like $e^{-1/(8\gamma)}$, which is not fast enough to compensate, so averaged over γ the second moment is infinite: formally $\sigma_A^2 = \infty$.

Small values of γ are rare, which is why the estimator still works in practice, but when they occur they produce extreme samples.

The outer variance shrinks with more missing features. For a GMM with diagonal covariances, the characteristic function of the missing features satisfies

$$\left|\phi_{x^m|x^o}(-uw_m)\right| \leq \exp\left(-\frac{u^2}{2} D_m\right), \qquad D_m := \min_k \sum_{j \in \text{mis}} w_j^2 \, \sigma_{kj}^2,$$

where D_m is the variance of the missing part of $w^T x$. When D_m is large, the integrand is nearly zero unless $|u|$ is small, so the inner expectation is close to zero for a wide range of γ and hardly varies with it: $\sigma_B^2 \approx 0$. More missing features increase D_m, so PCPG-MC gains relative to MC as the missing rate grows, while MC behaves the opposite way, since more missing features make $w^T x$ more random and increase σ_{MC}^2.

PCPG Estimator

Variance. Replacing both expectations with quadrature (Gauss–PG for the outer, Gauss–Hermite for the inner) makes the estimator fully deterministic:

$$\text{Var}[\widehat{p}_{\text{PCPG}}] = 0.$$

Its output is bounded, since the nodes γ_j are fixed positive numbers and the factor $e^{1/(8\gamma_j)}$ is a known finite constant for each of them. The amplification is mild, because the smallest Gauss–PG node is not close to zero ($\gamma_{\min} \approx 0.044$ for 20 nodes, giving $e^{1/(8\gamma_{\min})} \approx 17$).

Bias. Both expectations introduce a quadrature error. Gauss–Hermite with N nodes is exact for polynomials in u up to degree $2N-1$, and for analytic integrands its error decays super-algebraically in N. The Gauss–PG rule with M nodes is exact for polynomials in γ up to degree $2M-1$. At $M = 20$ and $N = 60$, both errors are numerically negligible for moderate $|w^T x|$. For large positive $w^T x$ the outer error is amplified by $e^{z/2}$, which the sign flip (Section 2.2.2) removes; with the flip, the outer rules fitted directly to the sigmoid (Appendix A.3.2) are more accurate than Gauss–PG with fewer nodes. The inner Gauss–Hermite error for large $|w^T x|$ is not affected by the flip. The infinite inner variance of PCPG-MC does not disappear completely, however; it turns into quadrature error, which grows with the scale of $z = w^T x$. Section 4.1.2 discusses when this error and the error of the outer rule become significant.

Comparison

Table 4.1 summarises the properties of the three estimators.

Table 4.1.: Summary of the three estimators. MC uses N samples, PCPG-MC M outer and L inner samples, PCPG M Gauss–PG and N Gauss–Hermite nodes. "CF" means the estimator requires evaluating $\phi_{x^m|x^o}$ rather than drawing samples from $p(x^m \mid x^o)$.

Estimator	**Variance**	**Bias**	**Needs**
MC	$\sigma^2_{\mathrm{MC}}/N \leq 1/(4N)$	0	samples
PCPG-MC	$\sigma^2_A/(ML) + \sigma^2_B/M$, with $\sigma^2_A = \infty$	0	CF
PCPG	0	quadrature error	CF

Bias. MC and PCPG-MC are exactly unbiased. PCPG introduces a quadrature error, which decays fast in the number of nodes and is negligible at the parameter choices used ($M = 20$ Gauss–PG and $N = 60$ Gauss–Hermite nodes), as long as $|w^T x|$ is moderate.

Variance. MC has the variance σ^2_{MC}/N, bounded by $1/(4N)$. PCPG-MC is unbiased as well, but its inner variance σ^2_A is formally infinite because of the factor $e^{1/(8\gamma)}$, so rare small values of γ produce extreme samples, and its outer variance σ^2_B only becomes small when many features are missing. PCPG has no variance at all: its error is pure bias from the quadrature.

When to use which. MC is the simplest choice when sampling from $p(x^m \mid x^o)$ is easy. PCPG-MC mainly serves as a reference: it shows that the identities behind PCPG are exact, but sampling the two expectations is not competitive. PCPG is the method we propose: it only needs the characteristic function, and it is deterministic, so repeated evaluations at the same point agree exactly and it can be differentiated, for example when training through it.

4.1.2. What Determines the Accuracy

The scale of z

The missing ratio

The number of features

4.2. Training with PCPG

4.2.1. Why the estimate looks better but accuracy doesn't

4.2.2. Why MC isn't much worse

4.2.3. Why isn't standardised data better in accuracy

4.2.4. Takeaway

4.3. Related Work

Here, we just focus on works that also try to calculate

$$\mathbb{E}_{P(x^m|x^o)}\left[\sigma(w^T x)\right]$$

but use a different approach.

4.3.1. Naive Conformant Learning (NaCL)

Khosravi et al. [12] first point out that this expectation is hard to compute in general: even with a naive Bayes feature distribution, it is NP-hard. They also note that the popular shortcut of mean imputation only matches the expectation when the classifier is linear *and* the features are independent — for the sigmoid it is always biased.

Their fix is to pick a feature distribution on which the expectation *is* cheap, namely a naive Bayes model over binary features. The trick is that naive Bayes and logistic regression are dual to each other: every naive Bayes induces a logistic regression, and many naive Bayes models induce the same one. They call a naive Bayes *conformant* with the given LR if both give the same class probabilities on fully observed inputs, and

learn such a distribution via geometric programming ("naive conformant learning"). Once this naive Bayes is fitted, the expected prediction is just a marginal inference query and runs in linear time.

Difference to us: they keep the feature distribution simple (naive Bayes on binary inputs) so the expectation is exact and cheap; we keep the feature distribution rich (a GMM or PC on continuous inputs) and instead rewrite the sigmoid with two identities. They also have to refit the feature distribution whenever the classifier changes, while ours is trained once and reused.

4.3.2. Joint Modelling via SAEM

Jiang, Josse, and Lavielle [13] assume a joint Gaussian model on the covariates, $x \sim \mathcal{N}(\mu, \Sigma)$, together with logistic regression on top. Their main contribution is on the training side: they fit (μ, Σ, β) jointly from training data that may itself contain missing values, using a stochastic-approximation EM algorithm with Metropolis–Hastings updates.

For prediction at test time, they simply do plain Monte Carlo: sample $x^{m,(s)} \sim p(x^m \mid x^o)$ from the fitted Gaussian and average,

$$p(y \mid x^o) \approx \frac{1}{S} \sum_{s=1}^{S} p\Big(y \mid x^o, x^{m,(s)}\Big)$$

This is exactly the MC baseline we compare against, just with a single Gaussian instead of a mixture as the density.

Difference to us: they sample the missing features and average the sigmoid; we never sample the missing features, but instead rewrite the sigmoid so that only the characteristic function of $p(x^m \mid x^o)$ is needed. Their feature distribution is also limited to a single Gaussian, while ours can capture multimodal shapes.

Appendix

Appendix A.

Intuitive Derivations

A.1. Bells as Waves

The Hubbard-Stratonovich identity writes a bell as an expectation of $\exp(-iu(z-c))$ over $u \sim \mathcal{N}(0,\gamma)$, with $c = \frac{1}{2\gamma}$. To see the waves, split each term of the expectation with Euler's formula $e^{-ix} = \cos x - i \sin x$:

$$\exp\left(-iu\left(z-c\right)\right) = \cos\Big(u(z-c)\Big) - i\sin\Big(u(z-c)\Big).$$

The sine is odd in u, and since u and $-u$ are equally likely under $\mathcal{N}(0,\gamma)$, it cancels and only the cosine remains:

$$\exp\left(\frac{1}{2}z - \frac{1}{2}\gamma z^2\right) = \exp\left(\frac{1}{8\gamma}\right)\mathbb{E}_{u\sim\mathcal{N}(0,\gamma)}\left[\cos\Big(u(z-c)\Big)\right].$$

A.2. The Characteristic Function as an Area

The characteristic function is used to compute $\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right]$. The integrand depends on x^m only through $z = w^T x$, so we can take the expectation over the distribution $p(z)$ of z instead. The real part of $\exp(-iu(z-c))$ is the wave $\cos(u(z-c))$ from Appendix A.1:

$$\mathrm{Re}\left[\mathbb{E}_{p(x^m|x^o)}\left[\exp\left(-iu\left(z-c\right)\right)\right]\right] = \mathbb{E}_{p(z)}\left[\cos\Big(u(z-c)\Big)\right] = \int \cos\Big(u(z-c)\Big)\,p(z)\,\mathrm{d}z.$$

This is the area under the wave times $p(z)$, where parts below zero count negatively. So for every frequency u, the characteristic function gives the signed area under the wave times $p(z)$.

A.3. Other Rules for the PG Integral

Besides Gauss–PG (Section 2.2.2), we consider three families of rules for the outer integral over $\gamma \sim \mathrm{PG}(1,0)$: trapezoid, Gauss–Legendre and Gauss quadrature, each with the nodes spaced in γ and in $\log\gamma$, and rules whose weights are fitted to the sigmoid. All rules except Gauss–PG need the density p of $\mathrm{PG}(1,0)$ pointwise. It has no closed form, but two alternating series, one converging fast for small and one for large γ, evaluate it to machine precision. All weights are normalised to sum to one.

A.3.1. Spaced in γ and in $\log\gamma$

Trapezoid. The composite trapezoid rule on $[0,T]$ with n equally spaced nodes γ_j and spacing h has the weights $w_j \propto c_j\, p(\gamma_j)\, h$, with $c_j = \frac{1}{2}$ at both ends and 1 otherwise. We use $T = 5$, above which $\mathrm{PG}(1,0)$ has a mass of only about $2 \cdot 10^{-11}$. With equal spacing, only few nodes fall where the mass is, around $\gamma \approx 0.1$ to 0.5.

Log-trapezoid. Substituting $\gamma = e^s$ turns the expectation into

$$\mathbb{E}_{\gamma\sim\mathrm{PG}(1,0)}[f(\gamma)] = \int_{-\infty}^{\infty} f(e^s)\, p(e^s)\, e^s \,\mathrm{d}s.$$

Applying the trapezoid rule with n equally spaced nodes s_j and spacing h gives nodes $\gamma_j = e^{s_j}$ and weights $w_j \propto p(\gamma_j)\,\gamma_j\, h$. Since p decays like $e^{-1/(8\gamma)}$ towards 0 and like $e^{-\pi^2\gamma/2}$ towards infinity, the range $\gamma \in [10^{-3}, 10]$ covers all of the mass. Equal spacing in $\log\gamma$ places many nodes at small γ, exactly where $e^{-\gamma z^2/2}$ lives for large $|z|$, and for smooth integrands that vanish at both ends the trapezoid rule converges exponentially in n.

For the sigmoid alone, this makes the log-trapezoid rule more accurate than Gauss–PG with the same number of nodes. Inside PCPG, however, every node comes with the factor $e^{1/(8\gamma_j)}$ of f, which multiplies the error of the inner expectation. At the smallest node $\gamma = 10^{-3}$ this factor is about 10^{54}, which overflows in floating point and makes the estimate useless. The smallest Gauss–PG node for $n = 20$ is $\gamma \approx 0.044$, where the factor is about 17.

Gauss–Legendre. The standard Gauss–Legendre nodes $x_j \in [-1,1]$ with weights v_j are mapped to $\gamma_j = T(x_j+1)/2$ on the same interval $[0,T]$, and the density goes into the weights, $w_j \propto v_j\, \frac{T}{2}\, p(\gamma_j)$. The rule is exact for polynomials times the density, which is not what we integrate, so it does not reach the convergence rate of a Gaussian rule for $\mathbb{E}[f(\gamma)]$.

Log-Gauss–Legendre. The same in $s = \log \gamma$, on the same range as the log-trapezoid rule: the Legendre nodes are mapped to $[\log 10^{-3}, \log 10]$, and the weights get the factor $p(e^s)\, e^s$ as there.

Log-Gauss–PG. The Gaussian quadrature rule in $s = \log \gamma$, with the distribution of s as the weight function, so it is exact for polynomials in s up to degree $2n - 1$. The moments of s have no closed form, so the Jacobi matrix is not built from moments but by the Lanczos (Stieltjes) procedure on a fine discretisation of the distribution of s, with 4000 trapezoid points on $[\log 10^{-4}, \log 20]$. Lanczos builds the orthonormal polynomials point by point, with reorthogonalisation so that it stays stable, and their recurrence coefficients form the Jacobi matrix. The nodes and weights then follow from its eigendecomposition as in the Golub–Welsch algorithm.

A.3.2. Fitting the Nodes to the Sigmoid

With the sign flip (Section 2.2.2), the rule is only evaluated for z around and below 0, and the error that matters is $\frac{1}{2}e^{z/2}\varepsilon(z^2/2)$ on that side. None of the rules above optimise this quantity, since they all discretise the $\mathrm{PG}(1, 0)$ distribution.

Since the rule enters the estimator linearly, the estimator computes exactly $\mathbb{E}[\hat{\sigma}(z)]$, and its error is the error of $\hat{\sigma}$ averaged over the distribution of z. Nothing in the rest of the method requires the nodes to represent $\mathrm{PG}(1, 0)$: HS and the characteristic function only need $\gamma_j > 0$, and the weights only need to be nonnegative. We can therefore choose the nodes and weights to approximate the sigmoid itself. On a grid of 400 log-spaced candidate nodes $\gamma_j \in [0.02, 8]$ and 1500 fitting points $z_r \in [-30, 5]$, we minimise a norm of the error curve

$$e_r = \tfrac{1}{2}\, e^{z_r/2} \sum_j w_j\, e^{-\gamma_j z_r^2/2} - \sigma(z_r)$$

over nonnegative weights that sum to one. We use three norms:

- **L2**, the sum of squared errors, solved by nonnegative least squares with the sum-to-one constraint as an extra row,
- **L1**, the sum of absolute errors, i.e. the area under the error curve,
- **max**, the largest absolute error, which makes the error curve equally high everywhere.

L1 and max are linear programs. All three keep only a few of the candidates, so the number of nodes is not chosen directly. The sum-to-one constraint makes the estimate exactly $\frac{1}{2}$ at $z = 0$. The margin up to $z = 5$ covers the part of the distribution of z that remains on the positive side after the flip, and the grid starts at $\gamma = 0.02$, so that $e^{1/(8\gamma)}$ in the inner integral stays below about 500.

This changes the view of the outer integral: the method approximates the sigmoid by functions of the form $\frac{1}{2}e^{z/2}e^{-\gamma z^2/2}$, whose expectations are tractable, and the PG identity is one particular, exact but infinite, choice of how to combine them. A finite rule only has to combine them well, not to represent the PG distribution. The only variants this does not apply to are the ones that sample γ from $\mathrm{PG}(1,0)$, since a fitted rule cannot be sampled from.

A.4. Variants of PCPG

Each of the two expectations of PCPG can be computed by quadrature or by Monte Carlo, which gives the four combinations in Table A.1. The variants with Monte Carlo use Algorithm 3 with the sampled nodes and the equal weights $\frac{1}{M}$ and $\frac{1}{N}$ in place of the quadrature nodes and weights, and without the factor $\frac{1}{\sqrt{\pi}}$ of Gauss–Hermite.

Variant	**Outer** (γ_j, w_j)	**Inner** (u_{ij}, ω_{ij})
PCPG-MC	$\gamma_j \sim \mathrm{PG}(1,0)$, $w_j = \frac{1}{M}$	$u_{ij} \sim \mathcal{N}(0, \gamma_j)$, $\omega_{ij} = \frac{1}{N}$
PCPG-GH	$\gamma_j \sim \mathrm{PG}(1,0)$, $w_j = \frac{1}{M}$	Gauss–Hermite: $u_{ij} = \sqrt{2\gamma_j}\, t_i$, $\omega_{ij} = \frac{\omega_i}{\sqrt{\pi}}$
PCPG-GPG	Gauss–PG rule	$u_{ij} \sim \mathcal{N}(0, \gamma_j)$, $\omega_{ij} = \frac{1}{N}$
PCPG	Gauss–PG rule	Gauss–Hermite: $u_{ij} = \sqrt{2\gamma_j}\, t_i$, $\omega_{ij} = \frac{\omega_i}{\sqrt{\pi}}$

Table A.1.: The four combinations of Monte Carlo and quadrature for the two expectations of PCPG. Hermite nodes (t_i, ω_i) as in Section 2.2.2, Gauss–PG nodes (γ_j, w_j) as in Section 2.2.2.

PCPG-GPG samples the inner expectation, whose variance is very high, so it performs much worse than PCPG in practice.

A.5. Why Large Positive z Is Hard

In (2.5), any error ε a rule makes in approximating $\mathcal{L}(z^2/2)$ enters the estimate of $\sigma(z)$ multiplied by $\frac{1}{2}e^{z/2}$. The nodes only see z^2, so this error is the same for z and $-z$, but its effect is not:

$$\left|\hat{\sigma}(z) - \sigma(z)\right| = \tfrac{1}{2}\, e^{z/2}\, \varepsilon(z^2/2).$$

For $z < 0$ the factor shrinks the error, for $z > 0$ it amplifies it exponentially. Staying at $\sigma(z) \approx 1$ for large positive z requires the sum of exponentials to reproduce the tail $\mathcal{L}(z^2/2) \approx 2e^{-z/2}$ almost exactly, which only rules with nodes very close to $\gamma = 0$ can do. Rules with few or no nodes there (trapezoid, Gauss–Legendre, Gauss–PG) have a

tail that is too small, so $\hat{\sigma}(z)$ falls back to 0 and the error approaches 1. A rule that puts more weight at tiny γ than PG$(1, 0)$ really has there does the opposite: its tail is too heavy, and $\hat{\sigma}(z)$ blows up like $e^{z/2}$. Only the rules spaced in $\log \gamma$ stay accurate without further help, and only with enough nodes.

A.6. Further Variants of the Estimation Experiment

The four variants of the comparison in Section 3.1 between the raw data with $C = 1$ and the standardised data with $C = 10^{-3}$. The setup and the layout of the figures are the same.

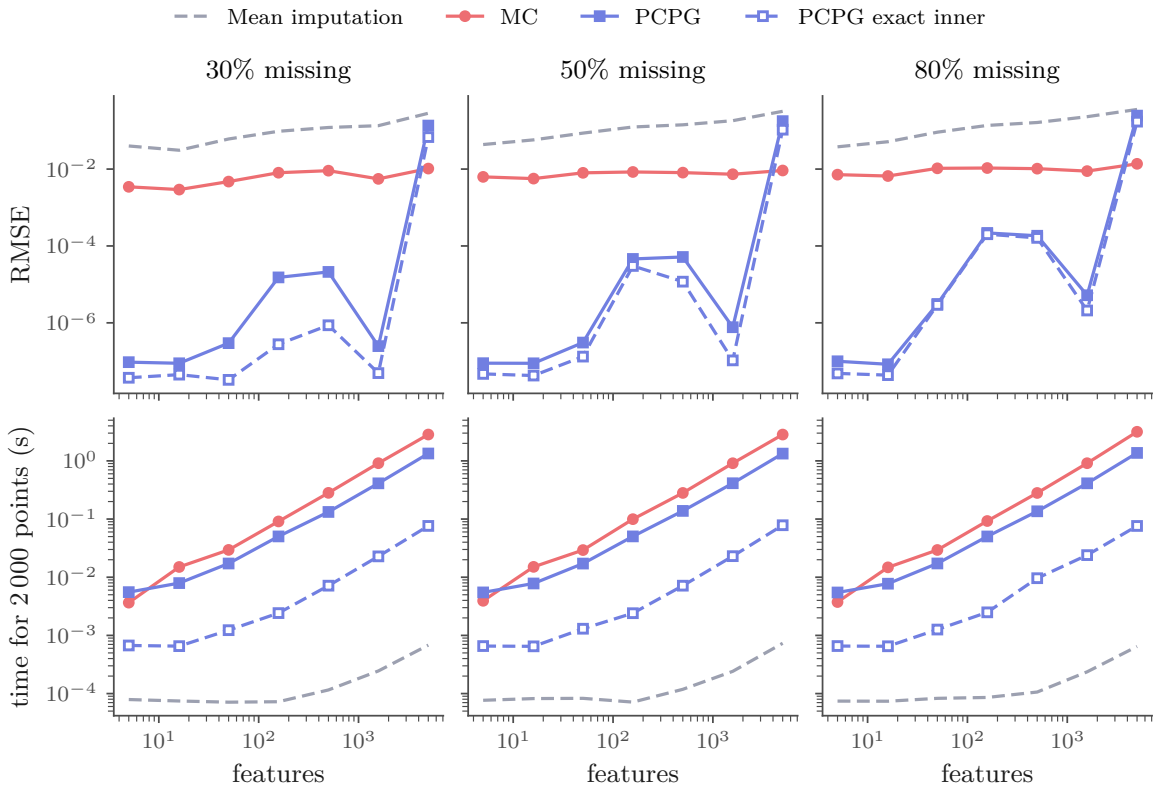


Figure A.1.: Raw data, weights of the classifier ($C = 1$) rescaled to $w/\|w\|$.

Rescaled weights. In Figure A.1 the weights are rescaled to unit norm. This does not help: at $d = 5000$ PCPG and PCPG exact inner still have an RMSE of about 10^{-1}. The reason is the data. The informative features of `make_classification` have a variance that grows with their number, up to about 900 at $d = 5000$, so $z = w^T x$ is spread widely even with $\|w\| = 1$.

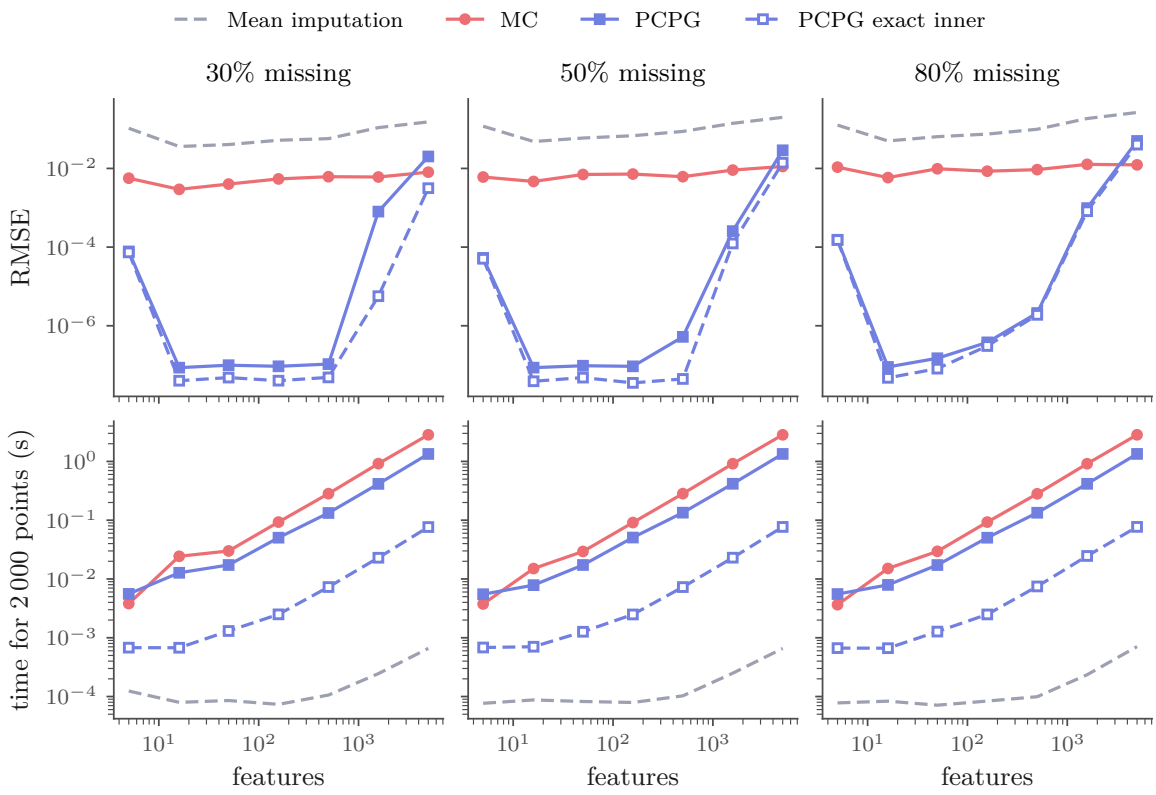


Figure A.2.: Standardised features, logistic regression with $C = 1$.

Standardised data. With standardised features and the trained weights (Figure A.2), the error at $d = 5000$ falls to $2 \cdot 10^{-2}$ to $5 \cdot 10^{-2}$, but it is still about as large as that of MC. Now the classifier is the cause: with $C = 1$ it overfits many features, and the standard deviation of z grows to about 12 at $d = 5000$ (Table 3.1).

Standardised data and rescaled weights. With both (Figure A.3), z has about the same scale for every d, and the error of PCPG no longer depends on d: it is about 10^{-7} for every number of features and missing rate, and about $3 \cdot 10^{-8}$ for PCPG exact inner. MC stays between $2 \cdot 10^{-3}$ and $6 \cdot 10^{-3}$. Rescaling w keeps every decision of the classifier, but it is no longer the trained classifier.

Standardised data, $C = 10^{-2}$. With $C = 10^{-2}$ (Figure A.4), the standard deviation of z stays below 4 (Table 3.1), and the error of PCPG stays below $2 \cdot 10^{-6}$ for most settings and reaches about 10^{-5} at $d = 5000$.

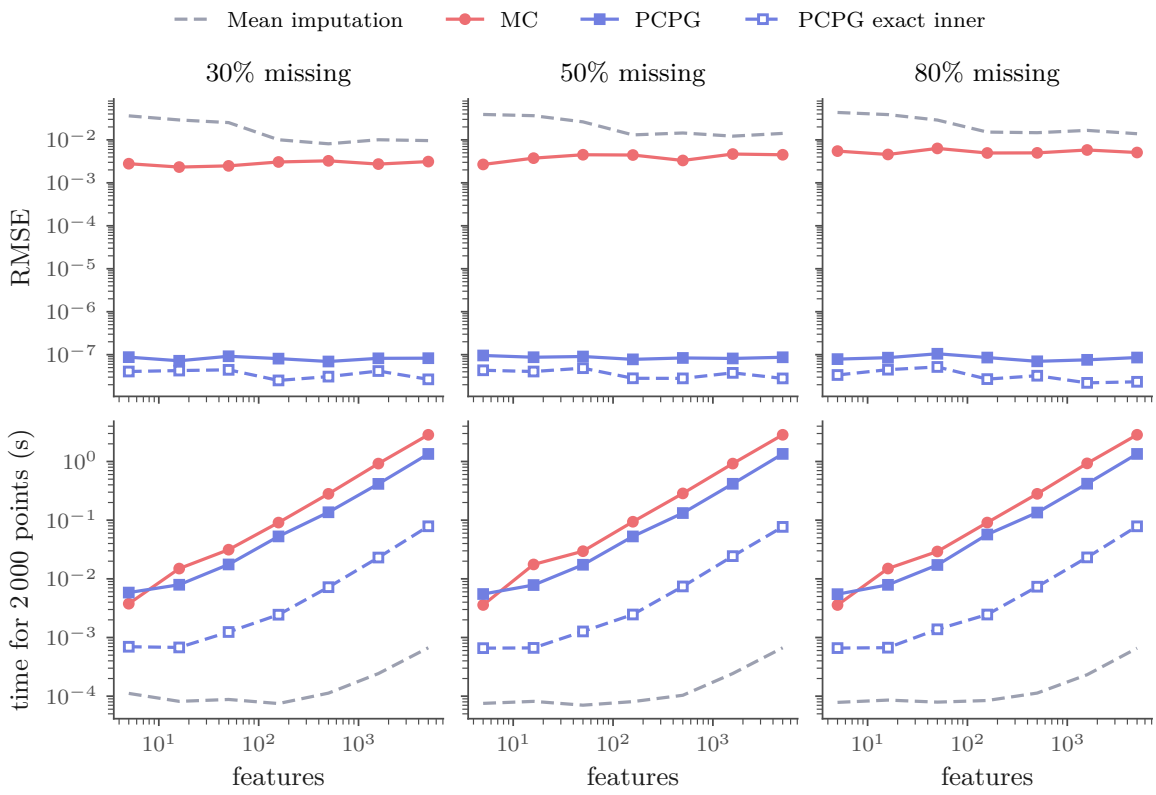


Figure A.3.: Standardised features, weights of the classifier ($C = 1$) rescaled to $w/\|w\|$.

A.7. Further Results of the Training Experiment

The accuracy on the incomplete test points of the training experiment in Section 3.2, with three prediction rules: every model with the exact expected prediction, every model with mean imputation (the missing features filled by their training means), and every model with its own rule as in Figure 3.5. The classifier trained on complete data is the yellow band. The differences are discussed in Section 4.2.

Accuracy on incomplete test points, raw data. With the exact expected prediction (Figure A.5, top), all classifiers reach almost the same accuracy at 30% and 50% missing. At 80% missing, the classifiers trained on the marginal likelihood are more accurate than the one trained on complete data, by 2.0 percentage points at 5 features (57.7% against 55.7%), 1.9 at 50 features and 1.4 at 100 features. With mean imputation at test time (middle), the classifiers trained on the marginal likelihood lose accuracy up to 20 features, by up to 3.4 percentage points (20 features, 80% missing). Only at 5 features with 80% missing do all classifiers gain, by 1.3 to 2.4 percentage points. From 50 features on, the two rules agree to within 0.3 percentage points for every classifier. With every model's own rule (bottom), the classifiers trained on the marginal

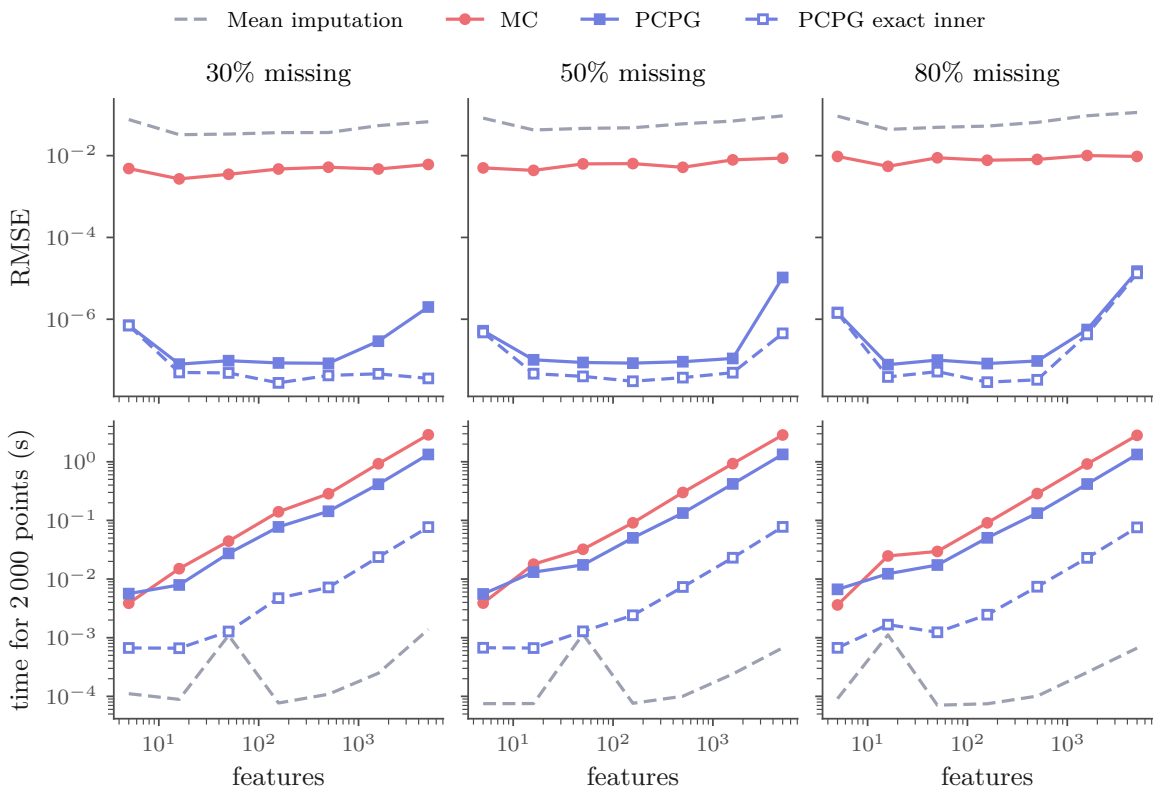


Figure A.4.: Standardised features, logistic regression with $C = 10^{-2}$.

likelihood are ahead of both baselines at 20 features, by 2.0 to 3.1 percentage points against complete data and 1.2 to 2.2 against mean imputation. At 10 features and from 50 features on, they are level with mean imputation and up to 1.8 percentage points ahead of complete data.

Accuracy on incomplete test points, standardised data. On the standardised data (Figure A.6), all classifiers are close with every rule. The largest difference to the complete-data classifier is 1.6 percentage points with the exact expected prediction, and 2.3 percentage points with the own rule, at 20 features and 80% missing, where the classifiers trained on the marginal likelihood are ahead. There is no consistent advantage for any model.

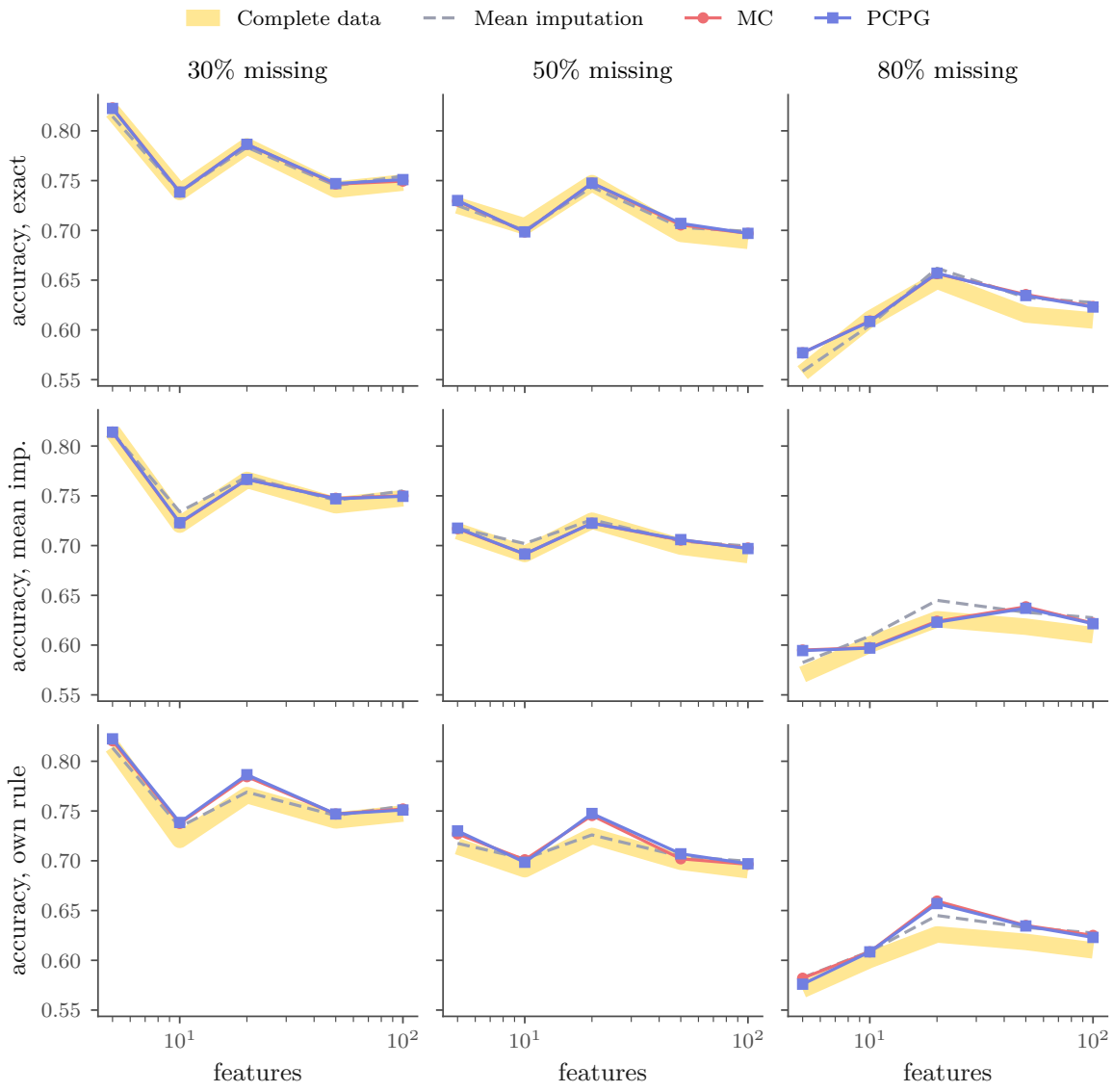


Figure A.5.: Accuracy on the incomplete test points, raw data with $C = 1$, against the number of features d. From top to bottom: every model predicts with the exact expected prediction, with mean imputation, and with its own rule. The reference is the classifier trained on complete data (yellow band).

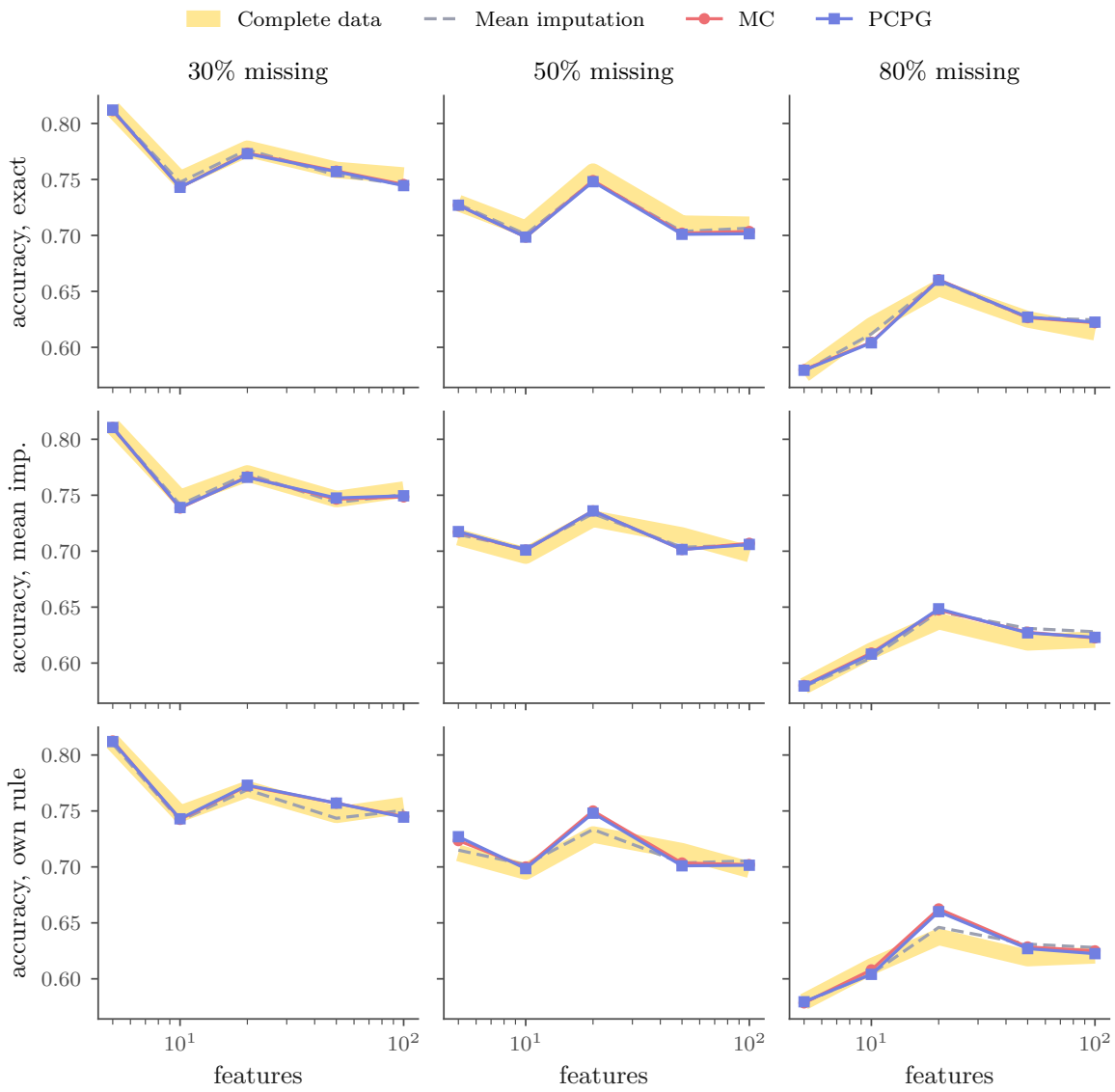


Figure A.6.: The same as Figure A.5, for the standardised data with $C = 10^{-3}$.

Bibliography

[1] YooJung Choi, Antonio Vergari, and Guy Van den Broeck. "Probabilistic Circuits: A Unifying Framework for Tractable Probabilistic Models." In: (Oct. 2020). URL: http://starai.cs.ucla.edu/papers/ProbCirc20.pdf (cit. on pp. 1–7).

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Berlin, Heidelberg: Springer-Verlag, 2006. ISBN: 0387310738 (cit. on pp. 3, 4).

[3] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: https://arxiv.org/abs/1406.2661 (cit. on p. 4).

[4] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114 [stat.ML]. URL: https://arxiv.org/abs/1312.6114 (cit. on p. 4).

[5] Tahrima Rahman, Prasanna Kothalkar, and Vibhav Gogate. "Cutset Networks: A Simple, Tractable, and Scalable Approach for Improving the Accuracy of Chow-Liu Trees." In: *Machine Learning and Knowledge Discovery in Databases*. Ed. by Toon Calders et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 630–645. ISBN: 978-3-662-44851-9 (cit. on p. 5).

[6] Doga Kisa et al. "Probabilistic sentential decision diagrams." In: *Fourteenth International Conference on the Principles of Knowledge Representation and Reasoning*. 2014 (cit. on p. 5).

[7] Hoifung Poon and Pedro Domingos. *Sum-Product Networks: A New Deep Architecture*. 2012. arXiv: 1202.3732 [cs.LG]. URL: https://arxiv.org/abs/1202.3732 (cit. on p. 5).

[8] Gene H. Golub and John H. Welsch. "Calculation of Gauss Quadrature Rules." In: *Mathematics of Computation* 23.106 (1969), pp. 221–230 (cit. on p. 8).

[9] Nicholas G Polson, James G Scott, and Jesse Windle. "Bayesian inference for logistic models using Pólya–Gamma latent variables." In: *Journal of the American statistical Association* 108.504 (2013), pp. 1339–1349 (cit. on p. 9).

[10] R. L. Stratonovich. "On a method of calculating quantum distribution functions." In: *Soviet Physics Doklady* 2 (1957), pp. 416–419 (cit. on p. 10).

[11] J. Hubbard. "Calculation of partition functions." In: *Physical Review Letters* 3.2 (1959), pp. 77–78. DOI: 10.1103/PhysRevLett.3.77 (cit. on p. 10).

[12] Pasha Khosravi et al. "What to expect of classifiers? Reasoning about logistic regression with missing features." In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*. 2019. URL: https://arxiv.org/abs/1903.01620 (cit. on p. 40).

[13] Wei Jiang, Julie Josse, and Marc Lavielle. "Logistic regression with missing covariates—Parameter estimation, model selection and prediction within a joint-modeling framework." In: *Computational Statistics & Data Analysis* 145 (2020), p. 106907. URL: https://arxiv.org/abs/1805.04602 (cit. on p. 41).